Numerical and simulation techniques in finance

Nicolas Langrené

Beijing Normal-Hong Kong Baptist University

2025 Fall Semester

4 Numerical simulation

Motivation

Financial option pricing: the fair-value price of a European option on an underlying $S = (S_t)_{0 \leq T}$ with final payoff $f(S_T)$ at time T can be represented mathematically as

$$\mathbb{E}[f(S_T)]$$

⚠ Note that for simplicity interest rates are assumed to be zero.

⚠ The **hedging strategy** should be factored in the price of the option. For simple models, this can be achieved by a *change of measure*, see Chapter 5 of [1] S. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.

Motivation

Here S_T is a random variable. The *distribution* of S_T depends on the *model* chosen for the process S. For most models, it is not possible to compute $\mathbb{E}[f(S_T)]$ exactly, so numerical approximations are needed.

In the previous chapter, we used the Feynman-Kac formula to write this expectation as the solution of a PDE, and proposed finite-different schemes to solve the PDE. In the next few chapters, we use an alternative numerical approximation called **Monte Carlo method**.

The idea is to approximate $\mathbb{E}[f(S_T)]$ by the **empirical average** $\frac{\mathbf{1}}{\mathbf{M}}\mathbf{\Sigma}_{\mathbf{m=1}}^{\mathbf{M}}\mathbf{f}(\mathbf{S}_{\mathbf{T}}^{\mathbf{m}})$ where $S_T^1, S_T^2, \ldots, S_T^M$ are independent and identically distributed (**iid**) **simulations** of the random variable S_T.

Probability theory: brief reminder (1/4)

Let X be a **random variable**. The **cumulative distribution function (CDF)** of X is the non-decreasing function F_X taking values in [0,1] defined by

$$F_X(x) := \mathbb{P}(X \leq x)$$

The **probability distribution function (PDF)** (a.k.a. **density**) f_X of a continuous random variable can be defined by the limit

$$f_X(x) := \lim_{h \to 0} \frac{\mathbb{P}(x - h < X \leq x + h)}{2h}$$

Note that $f_X(x) \geq 0$ for any x, and $\int_{-\infty}^{\infty} f_X(u) du = 1$. Both the PDF and the CDF can serve as the definition of the **distribution** of X.

Probability theory: brief reminder (2/4)

Using the definition of a density, the following holds

$$\mathbb{P}(X \in [a, b]) = \int_a^b f_X(u)du$$

for any $a < b$. In particular

$$F_X(x) = \int_{-\infty}^{x} f_X(u)du$$

and for a continuous random variable, $\mathbb{P}(X = x) = 0$ for any x.

Probability theory: brief reminder (3/4)

For any real function g, the following holds

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(u) f_X(u) du$$

In particular, the **moments** of the distribution of X are defined by

$$\mathbb{E}[X^p] = \int_{-\infty}^{\infty} u^p f_X(u) du$$

The first moment (p=1) is called the **mean** (a.k.a. **average**).

Probability theory: brief reminder (4/4)

The **variance** of the distribution of X is defined by

$$\mathbb{V}\mathrm{ar}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

The **standard deviation** of X is defined by $\sigma = \sqrt{\mathbb{V}\mathrm{ar}[X]}$

The **standardized moments** of the distribution of X are defined by

$$\mathbb{E}\left[\left(\frac{X - \mu}{\sigma}\right)^p\right] = \int_{-\infty}^{\infty} \left(\frac{u - \mu}{\sigma}\right)^p f_X(u)du$$

where $\mu = \mathbb{E}[X]$ is the mean of X, and σ is the standard deviation of X. The third standardized moment (p=3) is called the **skewness** of X, and the fourth standardized moment (p=4) is called the **kurtosis** of X.

4.1 Simulation of uniform random variables

Simulating random numbers

▷ Genuine **random numbers** can only be produced by physical and real-world processes, such as radioactive decay, or the outcomes from a fair casino roulette table.

▷ Random numbers from software might look genuinely random at first glance, yet they do come from a deterministic algorithm. Therefore, they are called **pseudorandom numbers**. Their main advantages are that they can be reproduced (which facilitates controlled and replicable experiments), and they can accommodate any type of distribution and properties.

Simulating random numbers

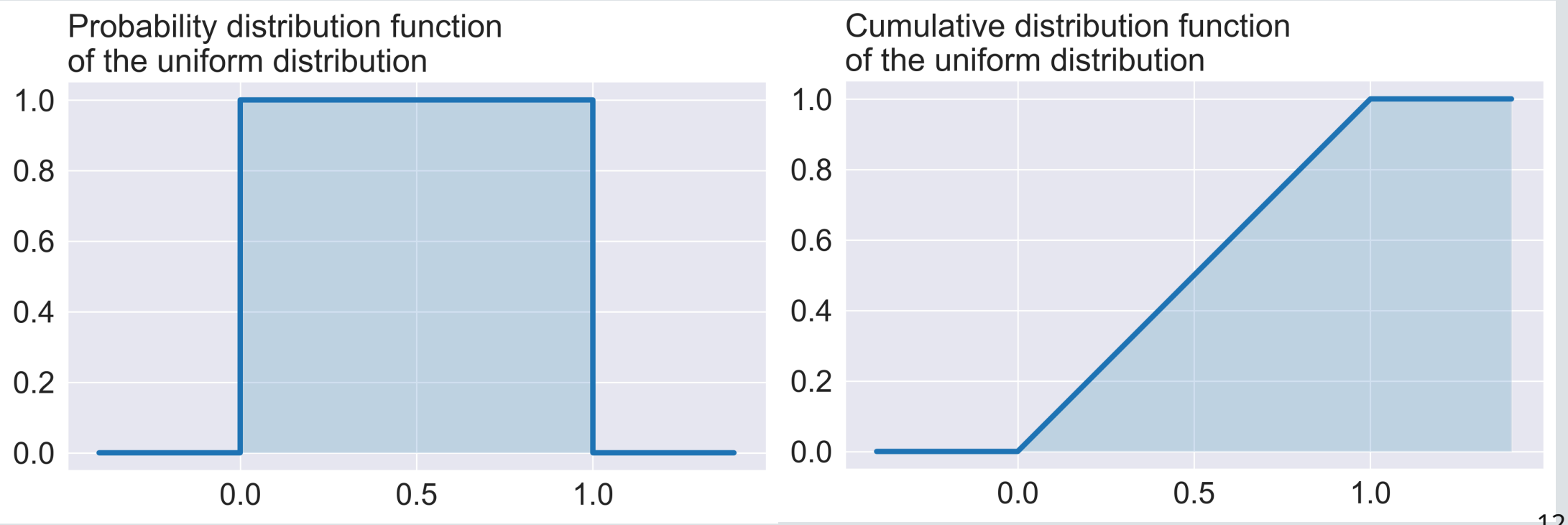
▷ **Quasirandom numbers** are constructed to maximize some goodness-of-fit measure (low discrepancy). They look too "perfect" to be random, but satisfy relevant theoretical properties as well. They are popular for numerical integration.

▷ **Bootstrap** methods resample from a given dataset. Their advantage is that no parametric distribution assumptions are required (*model-free* simulation) but they do require a representative set of available observations.

Uniform distribution

A uniform random variable U takes values in $[0, 1]$ and is such that

$$\mathbb{P}(a \leq U \leq b) = b - a \quad \text{for all } 0 \leq a \leq b \leq 1$$



Sequential congruential generators

A simple algorithm to generate a sequence of integers is the **linear congruential generator**

$$n_i = (an_{i-1} + c) \bmod m$$

where a, c, m and n_0 are integers. This method produces values $n_i \in \{0, \ldots, m-1\}$. To produce values in $[0, 1)$, one can simply define

$$u_i := \frac{n_i}{m}$$

If, for example, $m = 10^p$, we end up with values which can be considered real valued with a precision of p digits after the decimal point.

TABLE 6.1 Series of pseudorandom numbers from a linear congruential RNG, where $n_i = (a\, n_{i-1}) \mod 7$ and $n_0 = 1$ and different multipliers a.

	a										
	113	*114*	*115*	*116*	*117*	*118*	*119*	*120*	*121*	*122*	*123*
n_1	1	2	3	4	5	6	0	1	2	3	4
n_2	1	4	2	2	4	1	0	1	4	2	2
n_3	1	1	6	1	6	6	0	1	1	6	1
n_4	1	2	4	4	2	1	0	1	2	4	4
n_5	1	4	5	2	3	6	0	1	4	5	2
n_6	1	1	1	1	1	1	0	1	1	1	1
n_7	1	2	3	4	5	6	0	1	2	3	4
n_8	1	4	2	2	4	1	0	1	4	2	2
n_9	1	1	6	1	6	6	0	1	1	6	1
n_{10}	1	2	4	4	2	1	0	1	2	4	4

Linear congruential generators

Linear congruential generators generate **periodic sequences**. Since the parameters a, c and m are fixed, a certain realization n_{i-1} will always be followed by the same successor n_i.

At best, the periodic sequence can contain m unique numbers. For inappropriate choices for the values of a and c, the sequence could be much shorter as some integers within the range would never be drawn.

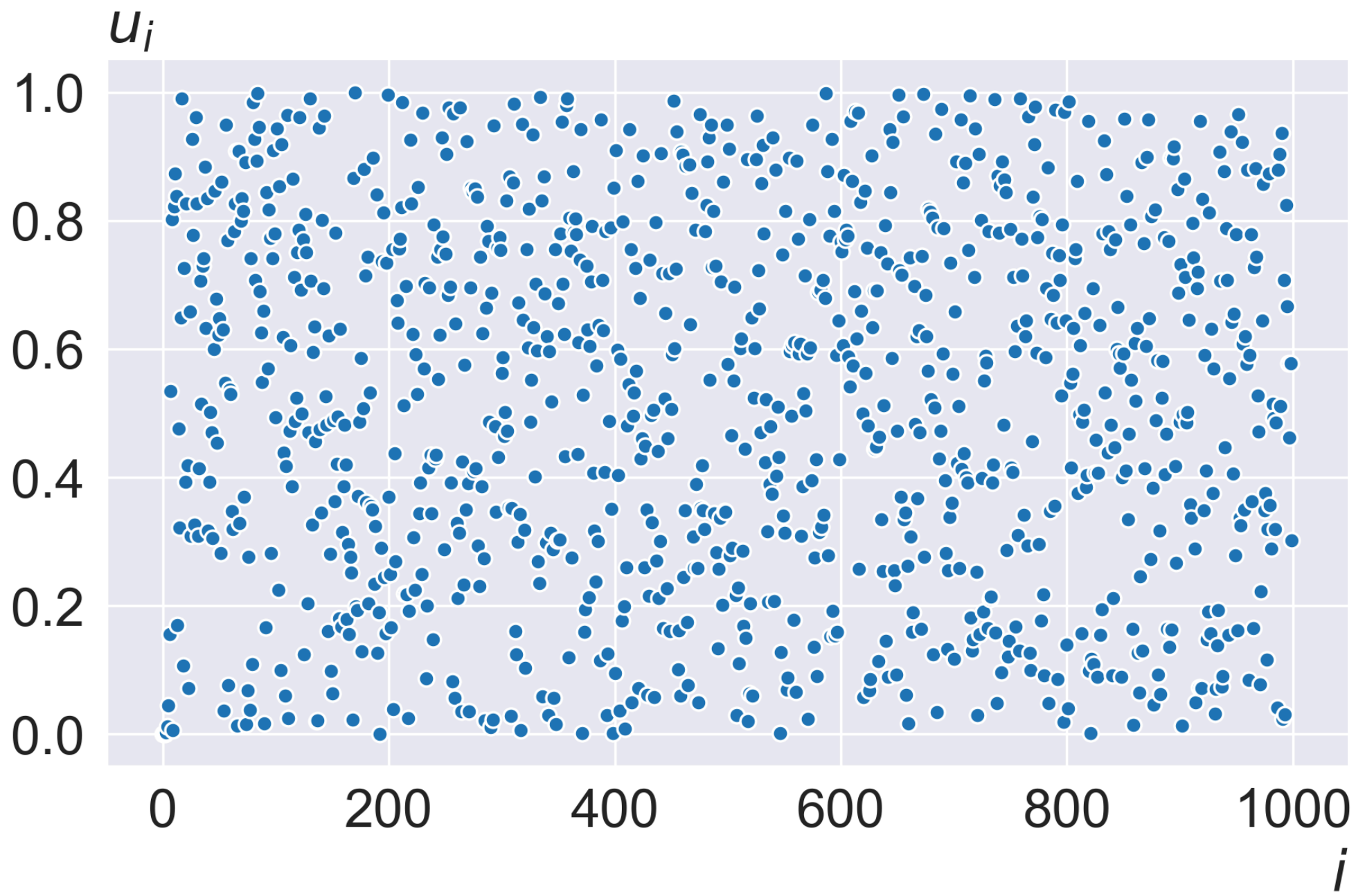
As a result, the statistical properties of congruential generators (can the u_i be distinguished from a true uniform random variable?) depend on the choice of a and c, and the larger m the better as it increases the maximum length of the periodic sequence.

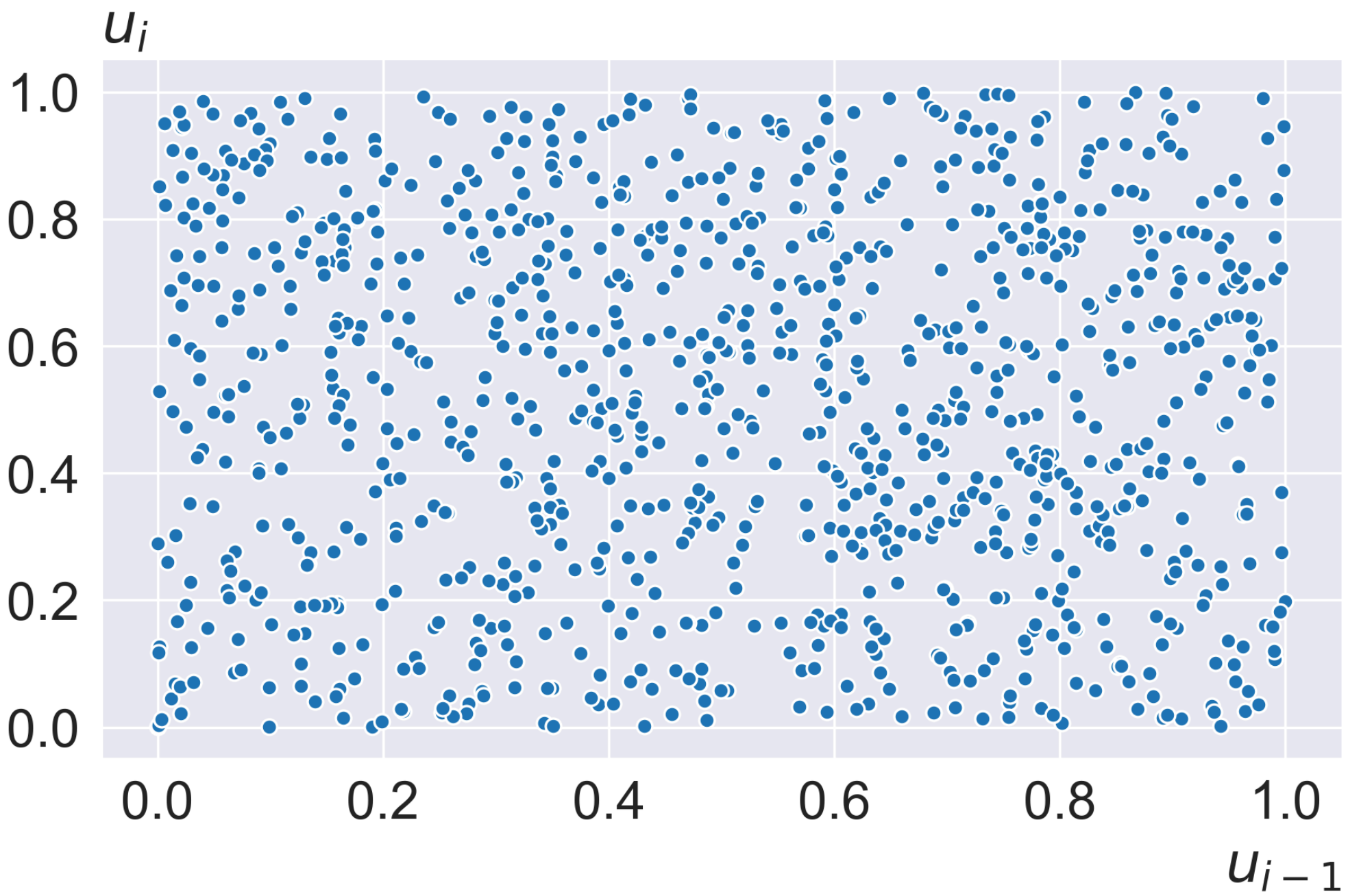
Example: IBM's RANDU

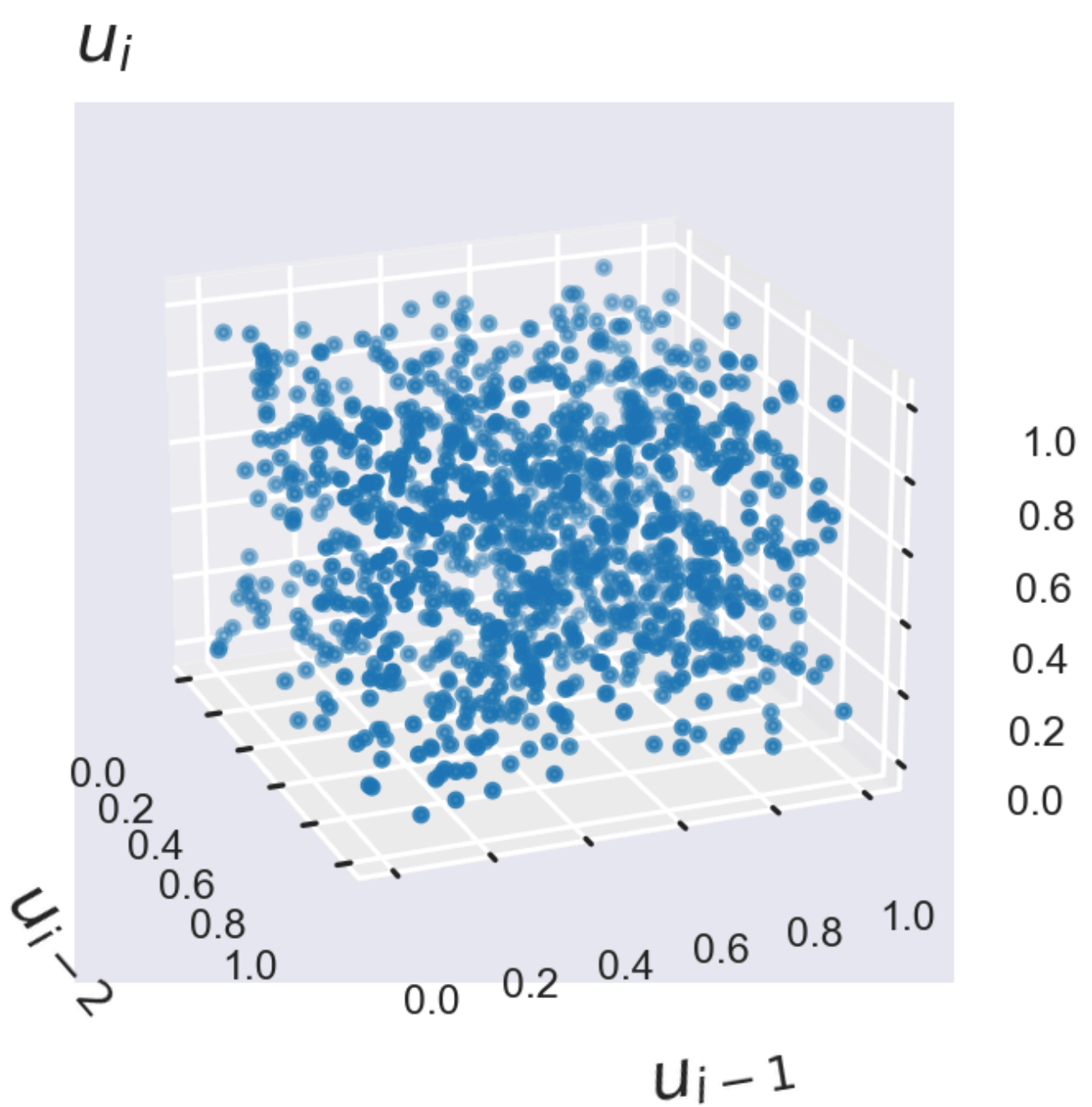
RANDU is an example of linear congruential generator. It was developed by IBM, and was primarily used in the 1960s and 1970s. It is defined by the following recurrence, where n_0 is chosen as an odd number:

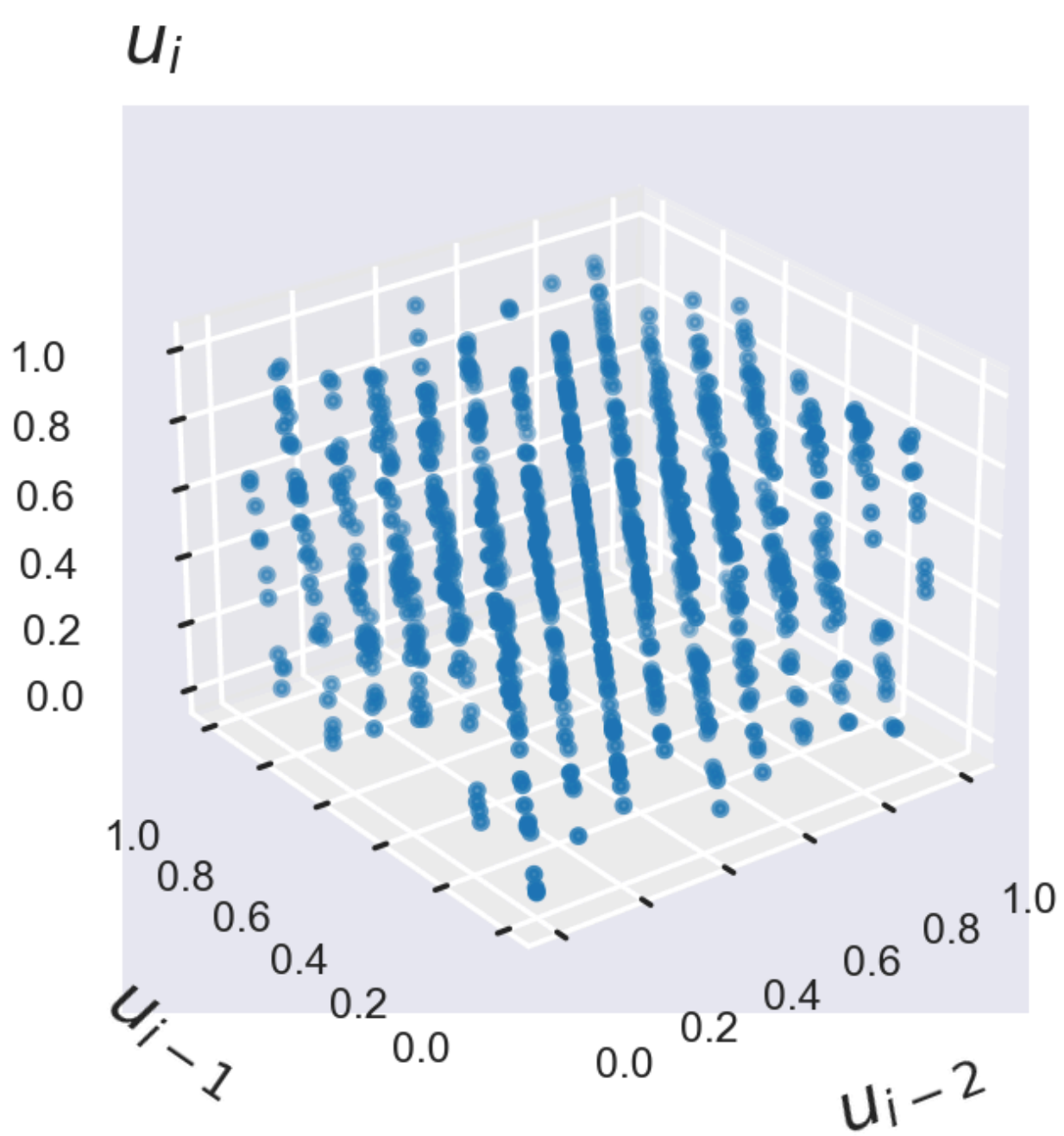
$$n_i = 65539 \times n_{i-1} \bmod 2^{31}$$

```
import numpy as np
a, c, m = 65539, 0, 2**31
N = 10**3
u = np.zeros(N,dtype=np.float64)
n = 1
for i in np.arange(N):
    n = (a*n+c)%m
    u[i] = n/m
```


u_i
1.0
0.8
0.6
0.4
0.2
0.0
0
200
400
600
800
1000
i


u_i
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-1}


u_i
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-2}
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-1}


u_i
1.0
0.8
0.6
0.4
0.2
0.0
1.0
0.8
0.6
0.4
0.2
0.0
u_{i-1}
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-2}

Observations

▷ The sequence u_i does seem to follow a uniform random distribution and to be independent of the previous values u_{i-1}. However, a careful check of the sequence u_i as a function of the two previous values u_{i-1} and u_{i-2} reveals that **all the points lie on one of 15 fixed parallel planes**. In other words, the sequence u_i is *not* i.i.d.

▷ The parameters $a = 65539 = 2^{16} + 3$, $c = 0$ and $m = 2^{31}$ have not been chosen for their statistical quality, but for their computational convenience. This yields a poor quality pseudorandom number generator.

▷ An example of parameterization with good statistical properties is $a = 1{,}664{,}525$, $c = 1{,}013{,}904{,}223$ and $m = 2^{32}$.

Mersenne Twister

The **Mersenne Twister** pseudorandom number generator was developed in 1997 by Makoto Matsumoto (松本 眞) and Takuji Nishimura (西村 拓士). The periodic sequence of the generator is chosen to be a **Mersenne prime number**. The most common version MT19937 is based on the Mersenne prime $2^{19937} - 1$. The details of the algorithm are rather complicated and therefore omitted.

Mersenne Twister has good statistical properties and is fast. It is a clear improvement over older sequential congruential generators. As a result, it became very popular and has been adopted as the default random number generator of many programming languages in the past 25 years. Until recently, Mersenne Twister was considered the state-of-the-art random number generator.

Permuted congruential generators

Permuted congruential generators (PCG) improve upon linear congruential generators (LCG) by *permuting* the state of the generator before producing one output. In addition, the output contains *fewer bits* than the generator state, which makes the resulting sequence less predictable. PCG is a parametric class of random number generators; one simple PCG example is the following:

$$
\begin{aligned}
n_i &= a n_{i-1} + c \\
p_i &= n_i >> (29 - (n_i >> 61)) \\
u_i &= \text{int32}(p_i)/2^{32}
\end{aligned}
$$

where a, c and n_0 are 64-bits integers.

Permuted congruential generators

$$\mathbf{n_i = an_{i-1} + c}$$

This linear transform is similar to the one of an LCG. There is no $\mathbf{mod}\, m$, which means that this transform is intended to generate an overflow. When an overflow occurs, only the lowest 64-bits are stored in n_i (the bits beyond that are discarded). This is mathematically the same as adding a $\mathbf{mod}\, m$ with $m = 2^{64}$.

$$\mathbf{p_i = n_i >> (29 - (n_i >> 61))}$$

This is the **permutation** step. The operator >> is a **bitwise left shift**. $n >> p$ is the same as an integer division of n by 2^p. Here the number of shifts depends on the highest three bits of n_i.

Permuted congruential generators

$$\mathbf{u_i} = \mathrm{int32}(\mathbf{p_i})/\mathbf{2^{32}}$$

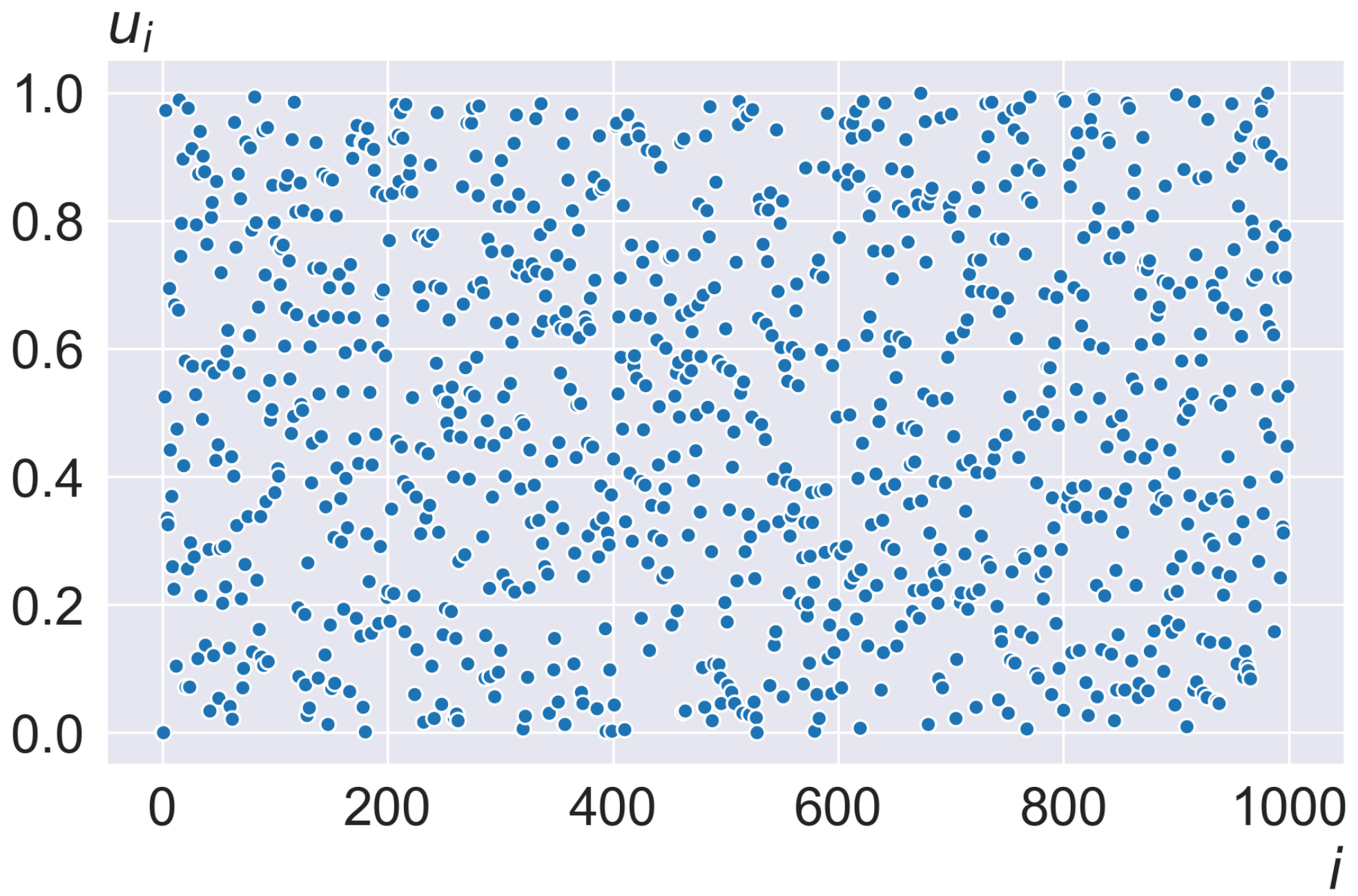
After the permutation, only the lowest 32 bits of p_i are returned as a pseudorandom integer between 0 and $2^{32} - 1$. Dividing by 2^{32} returns a pseudorandom float in $[0, 1)$.

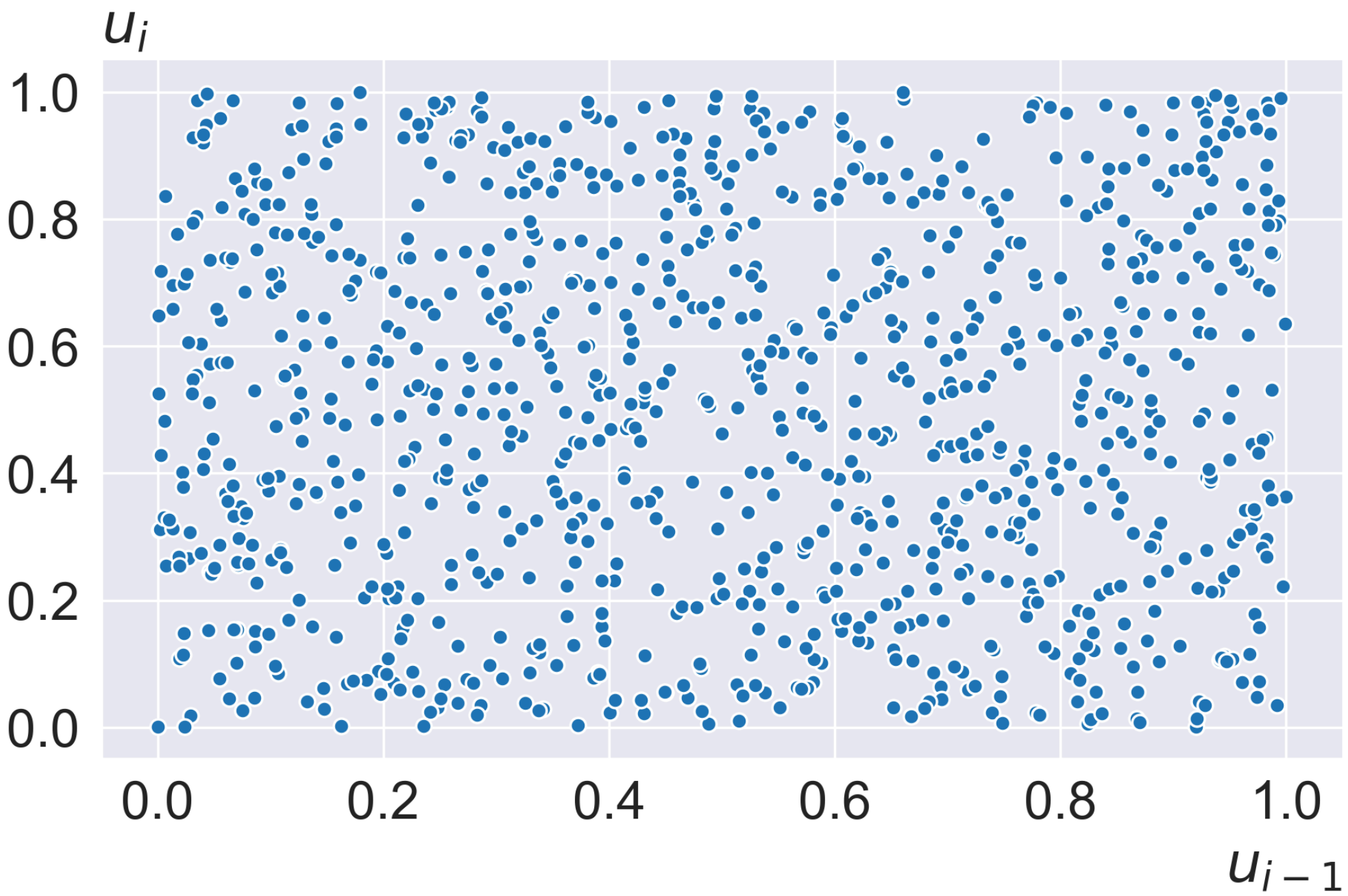
PCGs have been created by Pr. Melissa O'Neill in 2014. They enjoy better statistical properties than other state of the art random number generators, while being very fast (https://www.pcg-random.org/). PCGs are the new state-of-the-art in uniform random number generation, and are now the new default random number generator in NumPy.

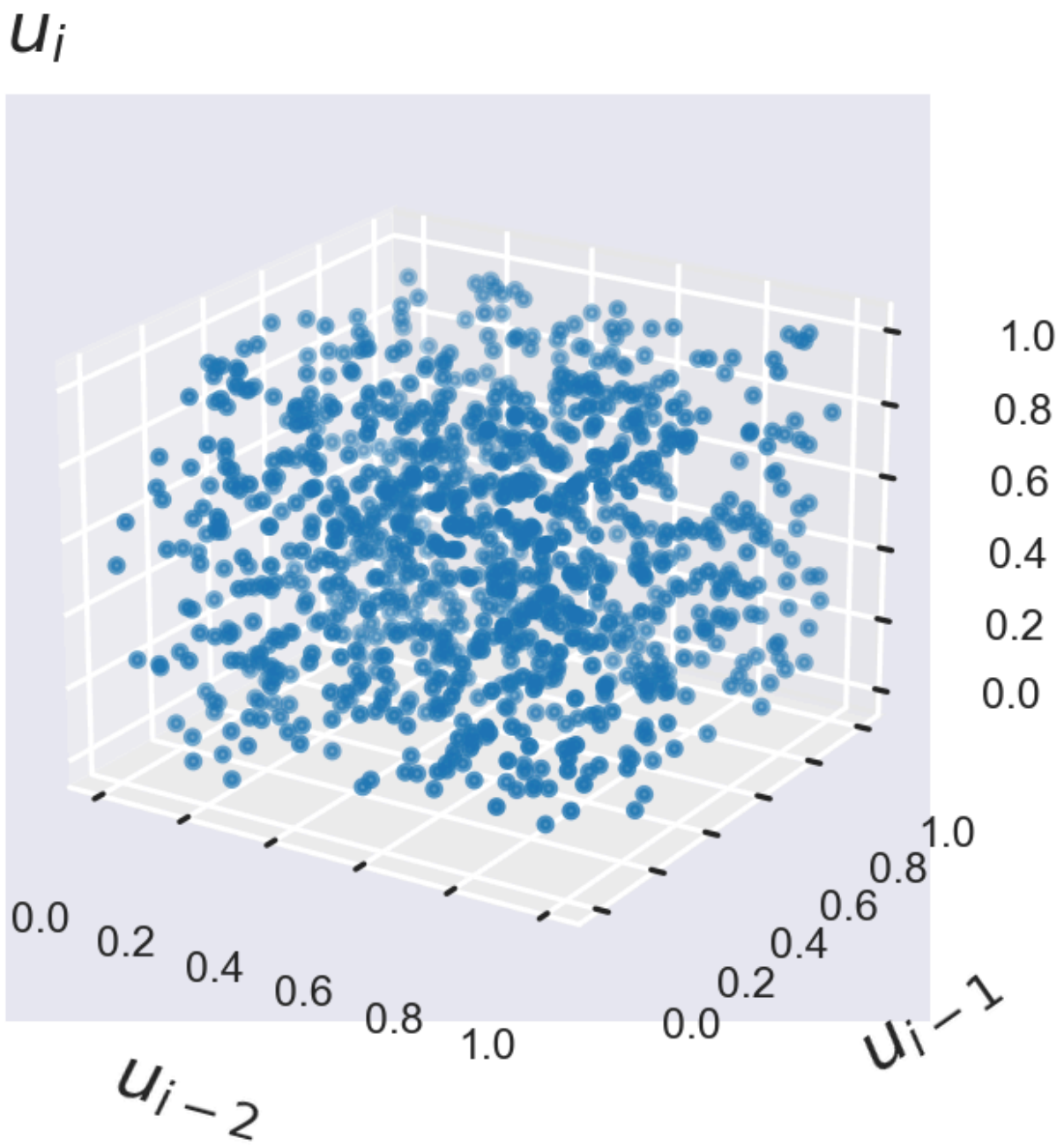
Permuted congruential generators

One possible Python implementation:

```
### PCG64
N = 10**3
u = np.zeros(N,dtype=np.float64)
a = np.uint64(1664525)
c = np.uint64(1013904223)
n = np.uint64(1)
for i in np.arange(N):
    n = a*n+c
    p = n//(2**(29 - n//(2**61)))
    u[i] = np.uint32(p)/(2**32)
```


u_i
1.0
0.8
0.6
0.4
0.2
0.0
0
200
400
600
800
1000
i


u_i
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-1}


u_i
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-2}
0.0
0.2
0.4
0.6
0.8
1.0
u_{i-1}

Uniform simulation with SciPy

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.uniform.html

```
from scipy.stats import uniform # uniform random variable
uniform.pdf( [-0.25, 0.25, 0.50, 0.75, 1.25] ) # pdf
## array([0., 1., 1., 1., 0.])
uniform.cdf( [-0.25, 0.25, 0.50, 0.75, 1.25] ) # cdf
## array([0.  , 0.25, 0.5 , 0.75, 1.  ])
uniform.ppf( [0.25, 0.50, 0.75] ) # inverse cdf
## array([0.25, 0.5 , 0.75])
uniform.rvs(size=[3,4]) # uniform simulations
## array([[0.66128092, 0.9869372 , 0.68011886, 0.74949296],
##        [0.07261969, 0.32097726, 0.50847636, 0.3228032 ],
##        [0.86837141, 0.37712304, 0.38601403, 0.00897607]])
```

Reproducible pseudorandom sequence

https://docs.scipy.org/doc/scipy/tutorial/stats.html

Since the sequence is pseudorandom, calling the **`rvs`** function several times in a row returns different numbers.

```
from scipy.stats import uniform
uniform.rvs(size=4) # rvs call 1
## array([0.29603734, 0.19345091, 0.17552225, 0.84425771])
uniform.rvs(size=4) # rvs call 2
## array([0.18966463, 0.60009041, 0.05532401, 0.77044007])
uniform.rvs(size=4) # rvs call 3
## array([0.40816565, 0.80803376, 0.4565929 , 0.61423157])
```

Reproducible pseudorandom sequence

In order to reproduce the same sequence, one can manually fix the **seed** (initial internal state) of the pseudorandom number generator.

```
from scipy.stats import uniform
from numpy.random import default_rng
rng = default_rng(12345) # set seed manually
uniform.rvs(size=4, random_state=rng) # rvs call 1
## array([0.22733602, 0.31675834, 0.79736546, 0.67625467])
uniform.rvs(size=4, random_state=rng) # rvs call 2
## array([0.39110955, 0.33281393, 0.59830875, 0.18673419])
rng = default_rng(12345) # reset seed to same value
uniform.rvs(size=4, random_state=rng) # rvs call 3, same as 1
## array([0.22733602, 0.31675834, 0.79736546, 0.67625467])
```

4.2 Simulation of non-uniform random variables

From uniform to non-uniform

The general idea is that one can transform simulations of a standard uniform random variable into simulations of other random variables with any probability distribution.

For example, simulations of a non-standard uniform variable $U \sim \mathcal{U}[a, b]$ on the interval $[a, b]$ can be obtained from simulations U_m, $m = 1, \ldots, M$ of a standard uniform variable $U \sim \mathcal{U}[0, 1]$ by the linear transform

$$a + (b - a)U_m$$

More generally two approaches are available:

- ▷ General techniques suitable for any probability distribution
- ▷ Specialized techniques suitable for specific probability distributions

Inversion method

Suppose that we want to simulate realizations of a random variable X with cumulative distribution $F(x) = \mathbb{P}(X \leq x)$. Suppose that F is strictly increasing and continuous. Then the inverse F^{-1} of F exists. If U is a standard uniform random variable, then

$$F^{-1}(U) \stackrel{d}{=} X$$

Indeed

$$\mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x) = \mathbb{P}(X \leq x)$$

In other words, the two random variables $F^{-1}(U)$ and X have the same distribution. This method is also known as **inverse transform sampling**.

Exponential distribution

A non-negative random variable is **exponentially distributed** with rate $\lambda > 0$ if it has the CDF

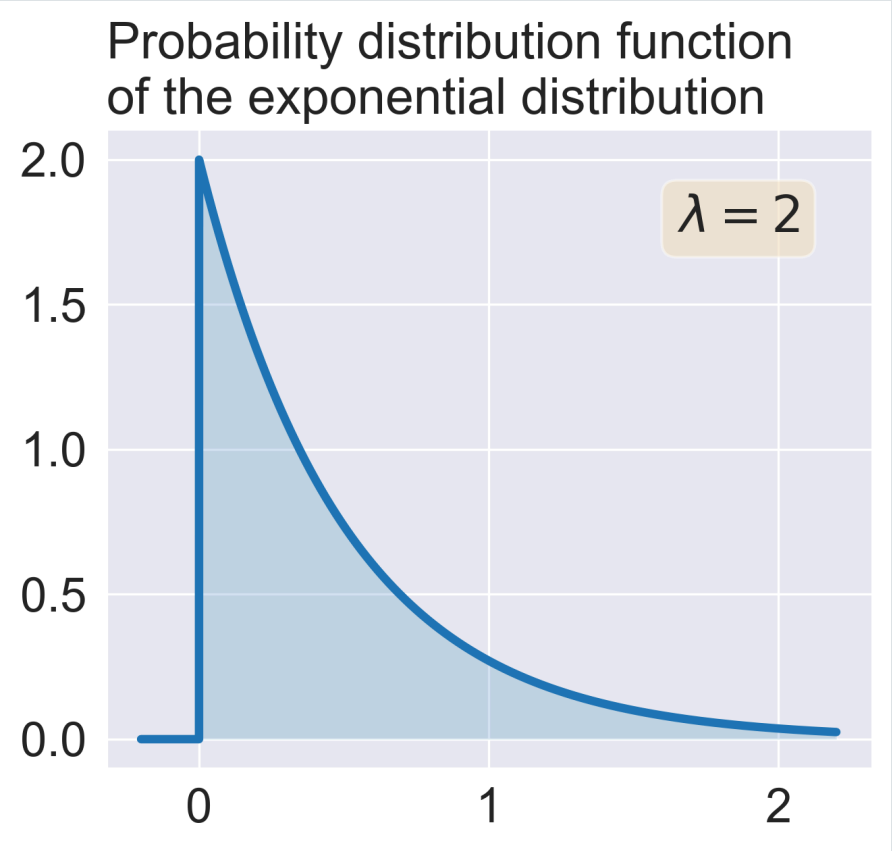
$$F(x) = 1 - e^{-\lambda x}$$

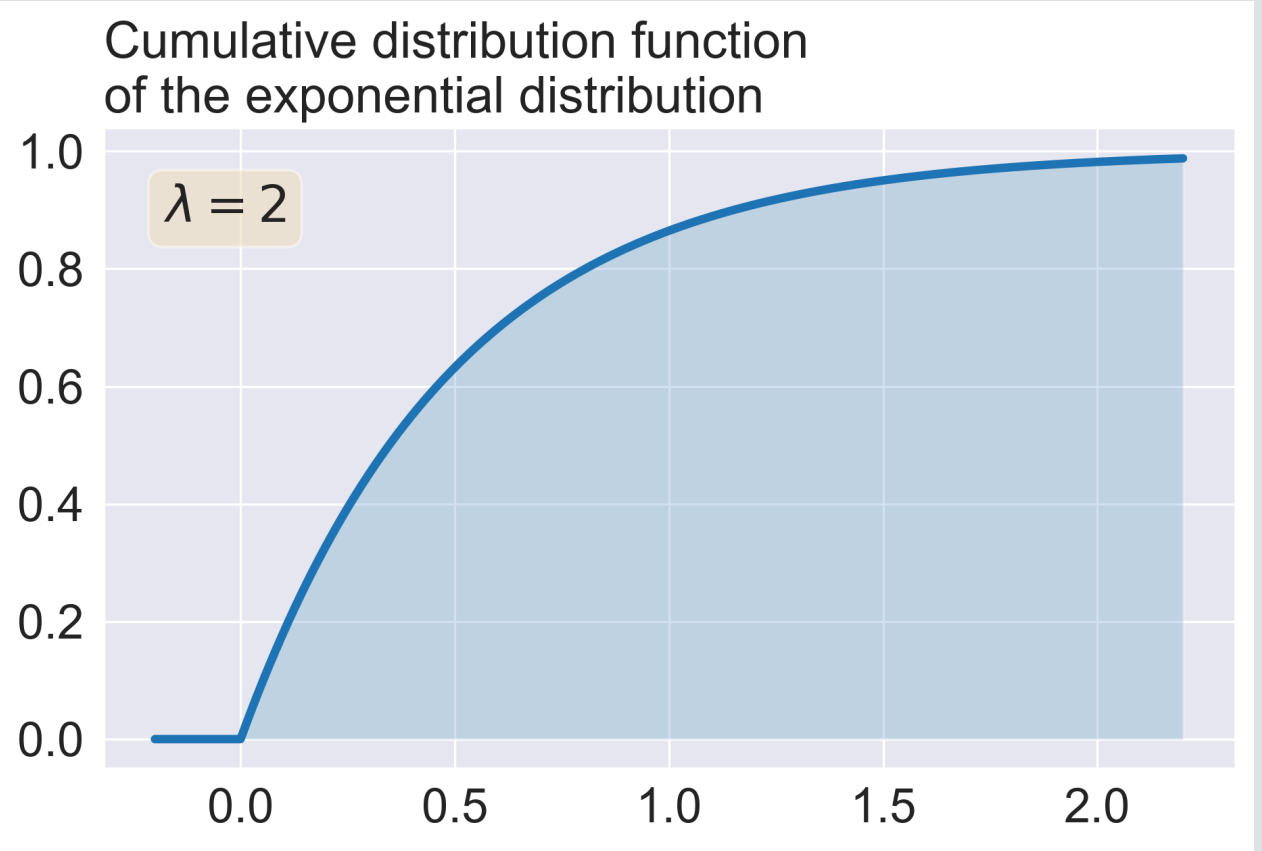
Its probability distribution function (PDF) is given by

$$f(x) = F'(x) = \lambda e^{-\lambda x} \mathbf{1}_{x \geq 0}$$

The exponential distribution appears as the waiting time between independent Poisson events (Poisson processes) and is suitable for measuring the life span of electronic devices or modelling radioactive decay.

Exponential distribution



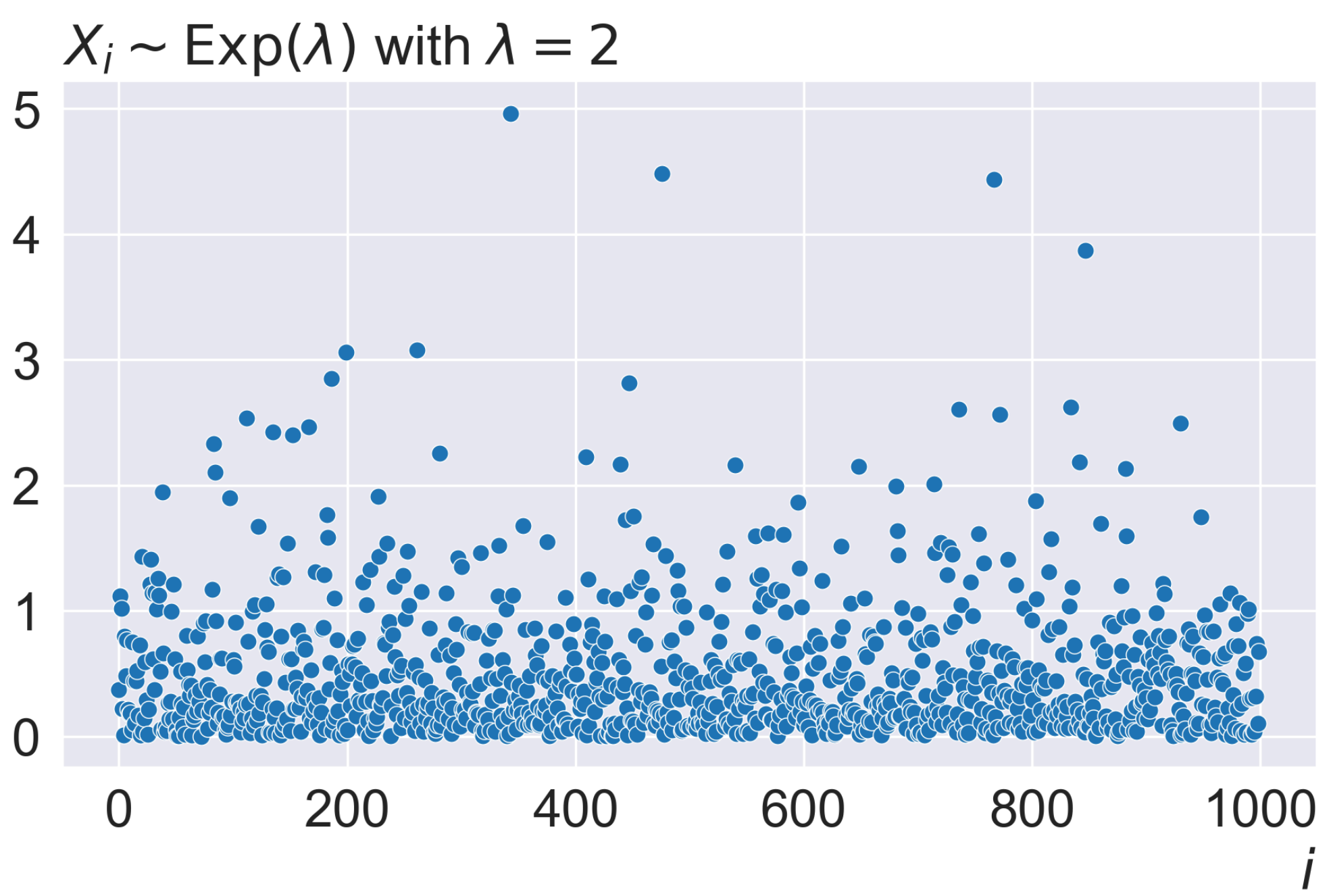


Simulate an exponential random variable

The exponential CDF $F(x) = 1 - e^{-\lambda x}$, $x \geq 0$ is invertible. Its inverse is given by

$$F^{-1}(u) = -\frac{\ln(1-u)}{\lambda}$$

According to the **inversion method**, the random variable $F^{-1}(U) = -\frac{\ln(1-U)}{\lambda}$, where U is a uniform random variable, follows an exponential distribution with rate λ. In practice, since U and $1 - U$ have the same distribution, one can simply simulate an exponential distribution with the formula $-\frac{\ln(U)}{\lambda}$.


$X_i \sim \text{Exp}(\lambda)$ with $\lambda = 2$
5
4
3
2
1
0
0
200
400
600
800
1000
i

Pareto distribution

The **Pareto distribution** with minimum value x_m and *tail index* α is defined by the PDF

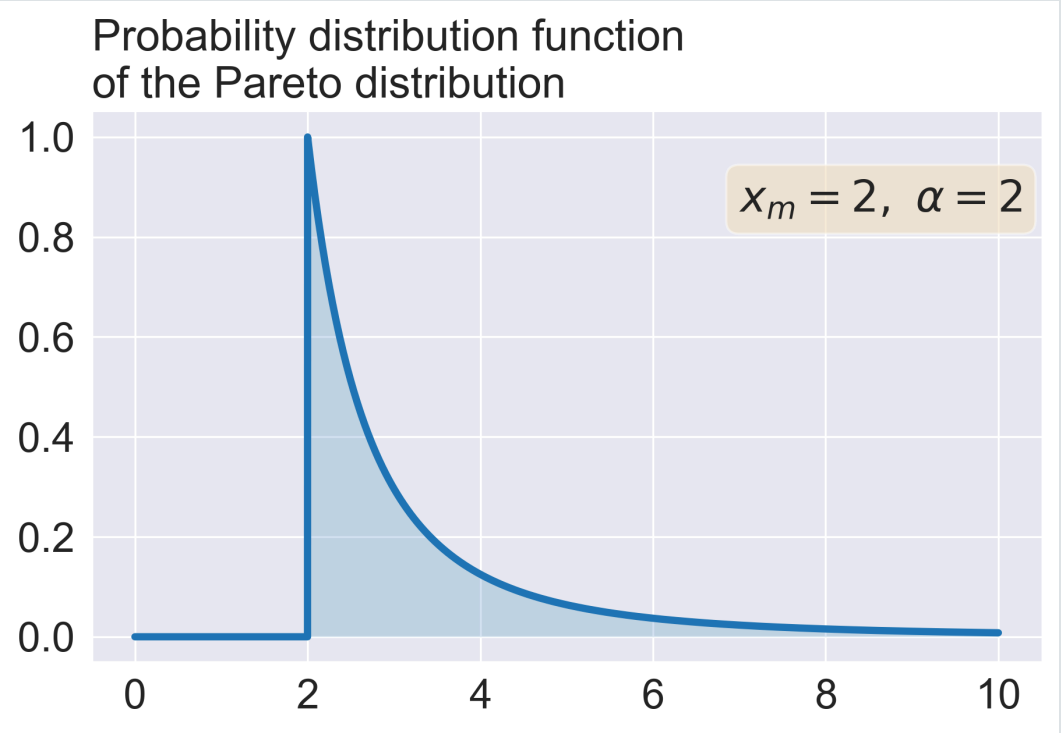
$$f(x) = \frac{\alpha x_m^\alpha}{x^{\alpha+1}}$$

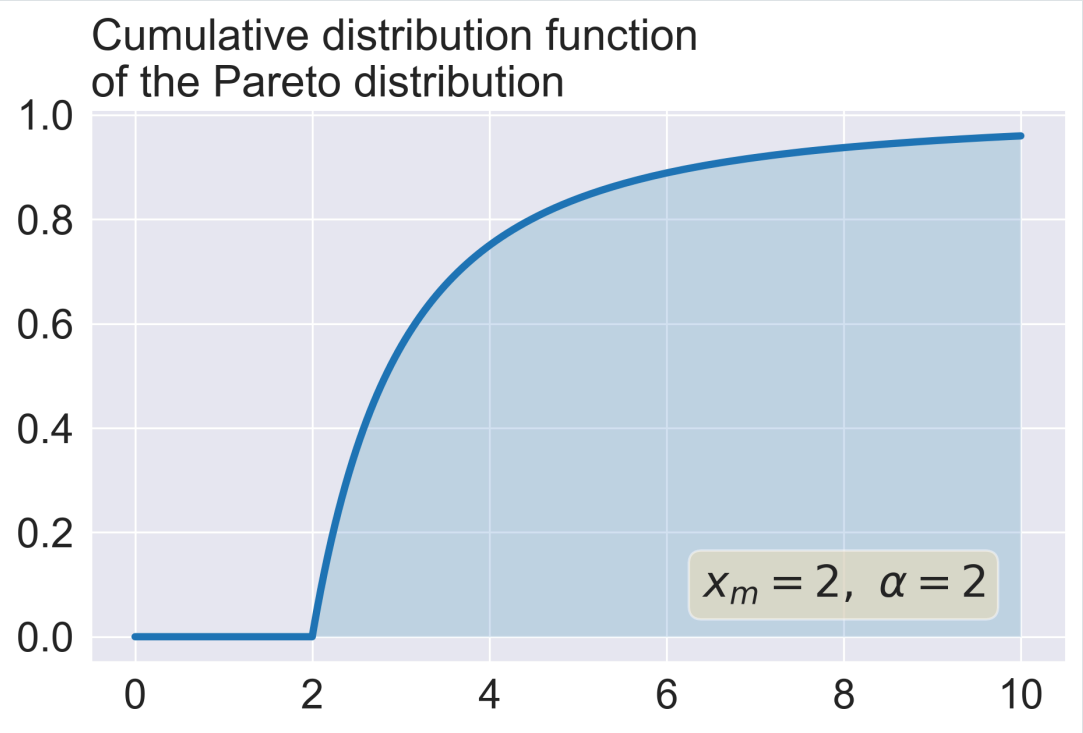
or equivalently by the CDF

$$F(x) = 1 - \left(\frac{x_m}{x}\right)^\alpha$$

This distribution was originally proposed by the Italian economist **Vilfredo Pareto** to describe the allocation of wealth among individuals. It has many other applications, in particular for modelling **extreme events** such as, for example, annual one-day maximum rainfall.

Pareto distribution

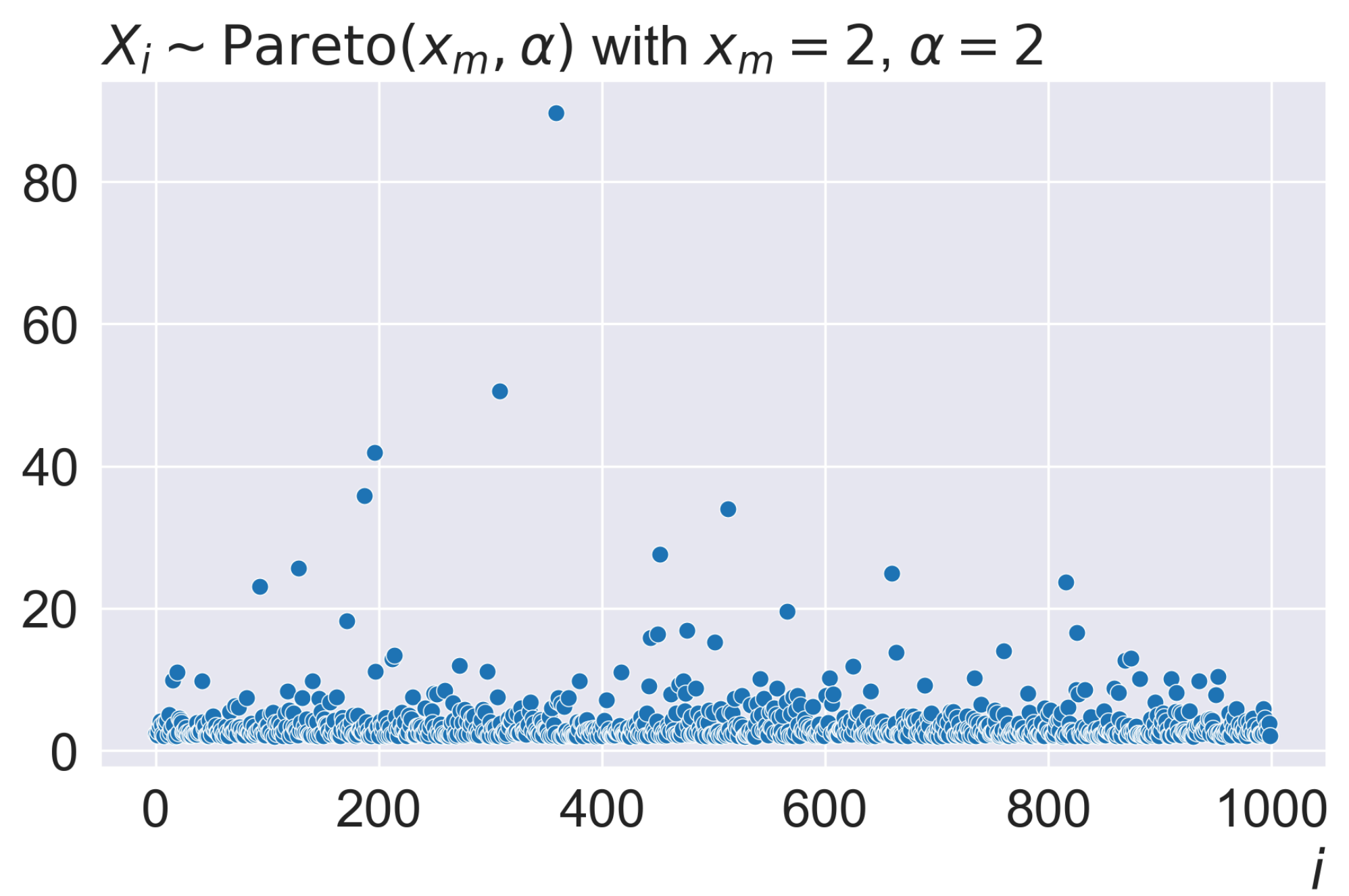


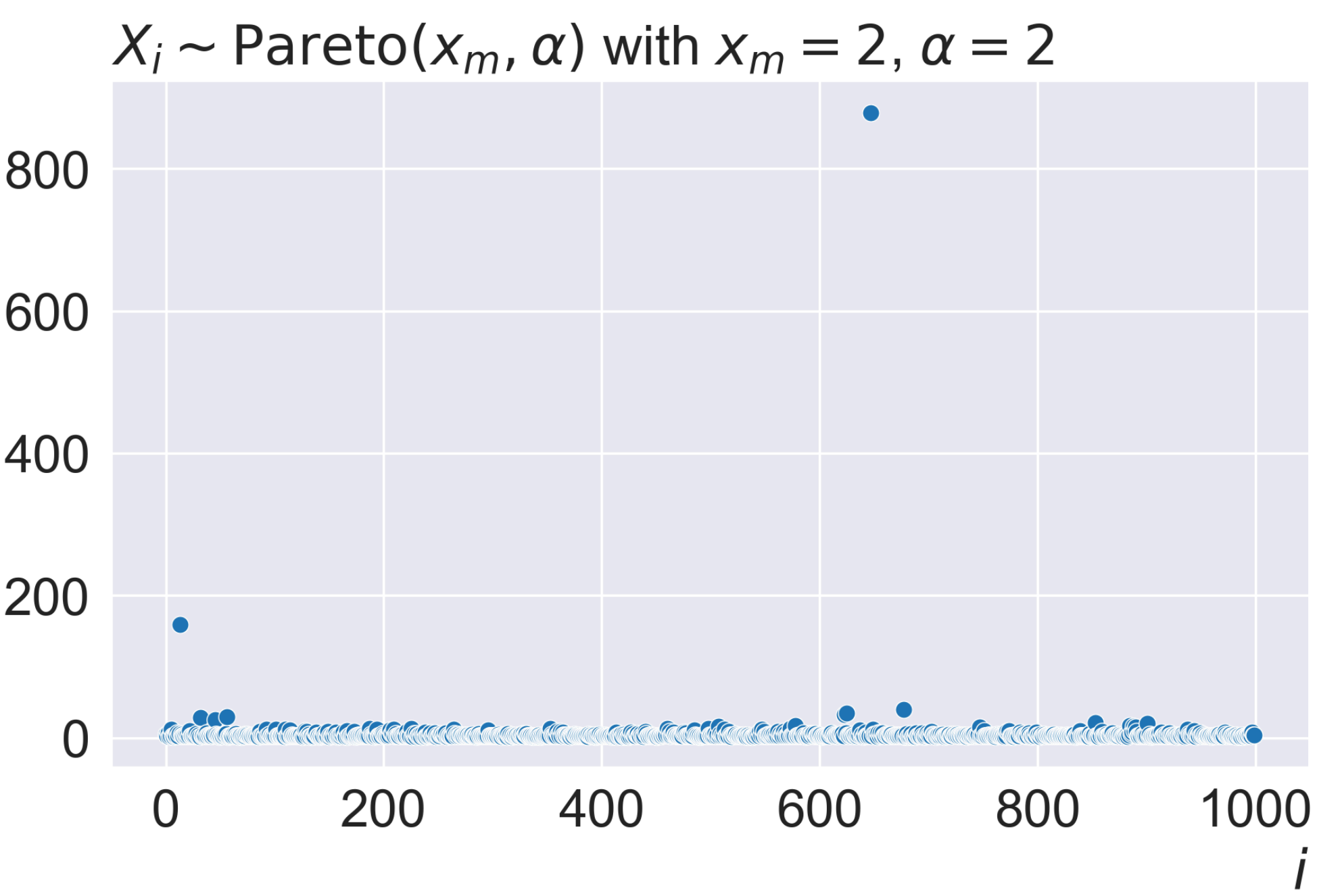


Simulate a Pareto random variable

The Pareto distribution is connected to the exponential distribution in the following way: if $X \sim \mathrm{Exp}(\alpha)$, then $Y = x_m \exp(X)$ is Pareto-distributed with minimum x_m and tail index α. This connection provides a straightforward way to simulate a Pareto random variable: simulate an exponential random variable X, then return $x_m \exp(X)$.

This is equivalent to applying the **inversion method** directly to the Pareto variable: the inverse of $F(x) = 1 - \left(\frac{x_m}{x}\right)^\alpha$ is $F^{-1}(u) = \frac{x_m}{(1-u)^{1/\alpha}}$, therefore the random variable $F^{-1}(U) = \frac{x_m}{(1-U)^{1/\alpha}}$ where $U \sim \mathcal{U}[0,1]$ is Pareto-distributed with minimum x_m and tail index α. Once again, U and $1-U$ have the same distribution so $x_m / U^{1/\alpha}$ is also Pareto-distributed.


$X_i \sim \text{Pareto}(x_m, \alpha)$ with $x_m = 2, \alpha = 2$
80
60
40
20
0
0
200
400
600
800
1000
i


$X_i \sim \text{Pareto}(x_m, \alpha)$ with $x_m = 2, \alpha = 2$
800
600
400
200
0
0
200
400
600
800
1000
i

Gaussian distribution

The *standard* **Gaussian distribution** (a.k.a. **normal distribution**) $\mathcal{N}(0, 1)$ with mean 0 and variance 1 is defined by the PDF

$$\phi(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

More generally, the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ with mean μ and variance σ^2 is defined by the PDF

$$\phi_{\mu,\sigma}(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Gaussian distribution

The Gaussian CDF $\Phi(x) = \int_{-\infty}^{x} \phi(u)du$ does not admit any explicit formula. Instead, it has to be approximated by numerical integration. One possible formula is the following (for $x < 0$, use $\Phi(x) = 1 - \Phi(-x)$)

Algorithm 2.10 Approximation of the standard normal c.d.f.

This function gives approximate values for the standard normal distribution:

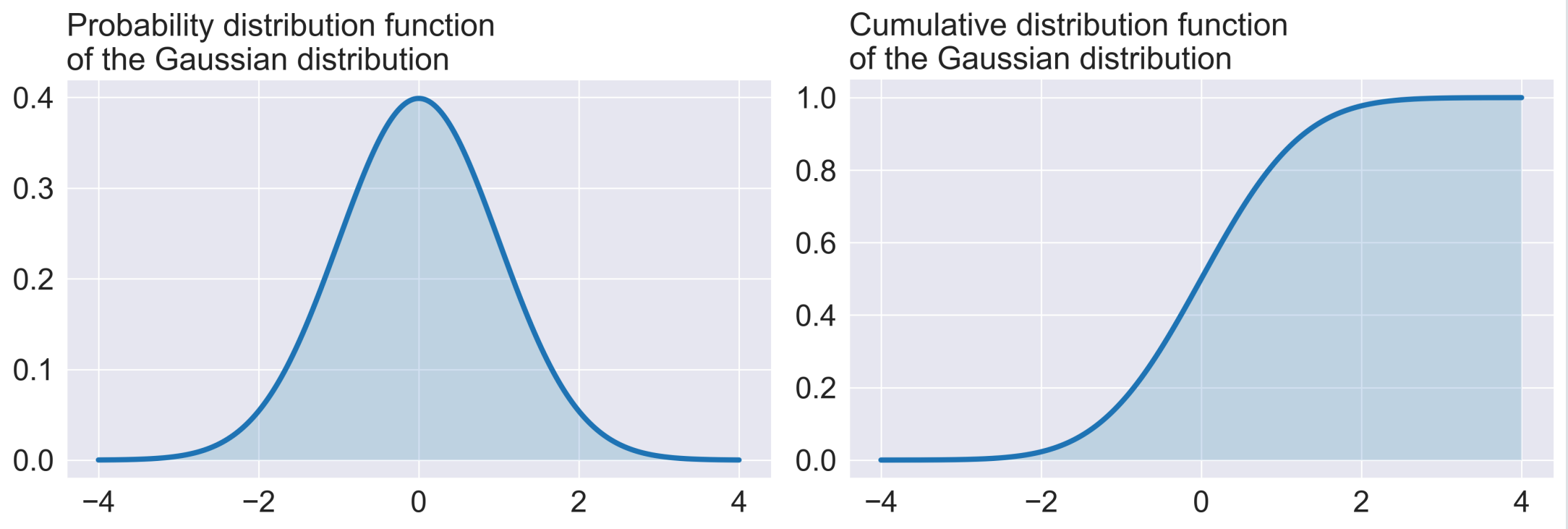
$$d := (0.0498673470, 0.0211410061, 0.0032776263, \\ 0.0000380036, 0.0000488906, 0.0000053830)$$

For $x \in [0, \infty)$ approximate

$$\Phi(x) \approx 1 - 0.5\left(1 + d_1 x + d_2 x^2 + d_3 x^3 + d_4 x^4 + d_5 x^5 + d_6 x^6\right)^{-16}$$

This approximation has a maximum relative error of $1.5 \cdot 10^{-7}$.

Gaussian distribution



Simulate a Gaussian random variable

We could simulate Gaussian random variables using the inversion method; however, the Gaussian CDF Φ and its inverse Φ^{-1} are not analytical functions: they need to be approximated numerically. For example, Φ^{-1} can be approximated by the **Beasley-Springer-Moro algorithm**

Algorithm 2.11 Beasley-Springer-Moro algorithm for the inverse standard normal

Let

$$
\begin{aligned}
a &:= (2.50662823884, -18.61500062529, 41.39119773534, -25.44106049637) \\
b &:= (-8.47351093090, 23.08336743743, -21.06224101826, 3.13082909833) \\
c &:= (0.337475482272614 7, 0.976169019091718 6, 0.160797971491820 9, \\
&\quad 0.027643881033386 3, 0.003840572937360 9, 0.000395189651191 9, \\
&\quad 0.000321767881768, 0.000000288816736 4, 0.000000396031518 7)
\end{aligned}
$$

Simulate a Gaussian random variable

1. For $u \in [0.5, 0.92]$ approximate

$$\Phi^{-1}(u) \approx \frac{\sum_{n=0}^{3} a[n]\left(u - \frac{1}{2}\right)^{2n+1}}{1 + \sum_{n=0}^{3} b[n]\left(u - \frac{1}{2}\right)^{2n}}$$

2. For $u \in (0.92, 1)$ approximate

$$\Phi^{-1}(u) \approx \sum_{n=0}^{8} c[n]\left(ln\left(-ln\left(1-u\right)\right)\right)^{n}$$

This approximation has a maximum absolute error of 3×10^{-9} over the range $[\Phi(-7), \Phi(7)]$.

For $u \in [0, 0.5)$, use $\Phi^{-1}(u) = -\Phi^{-1}(1-u)$

The Box-Muller method

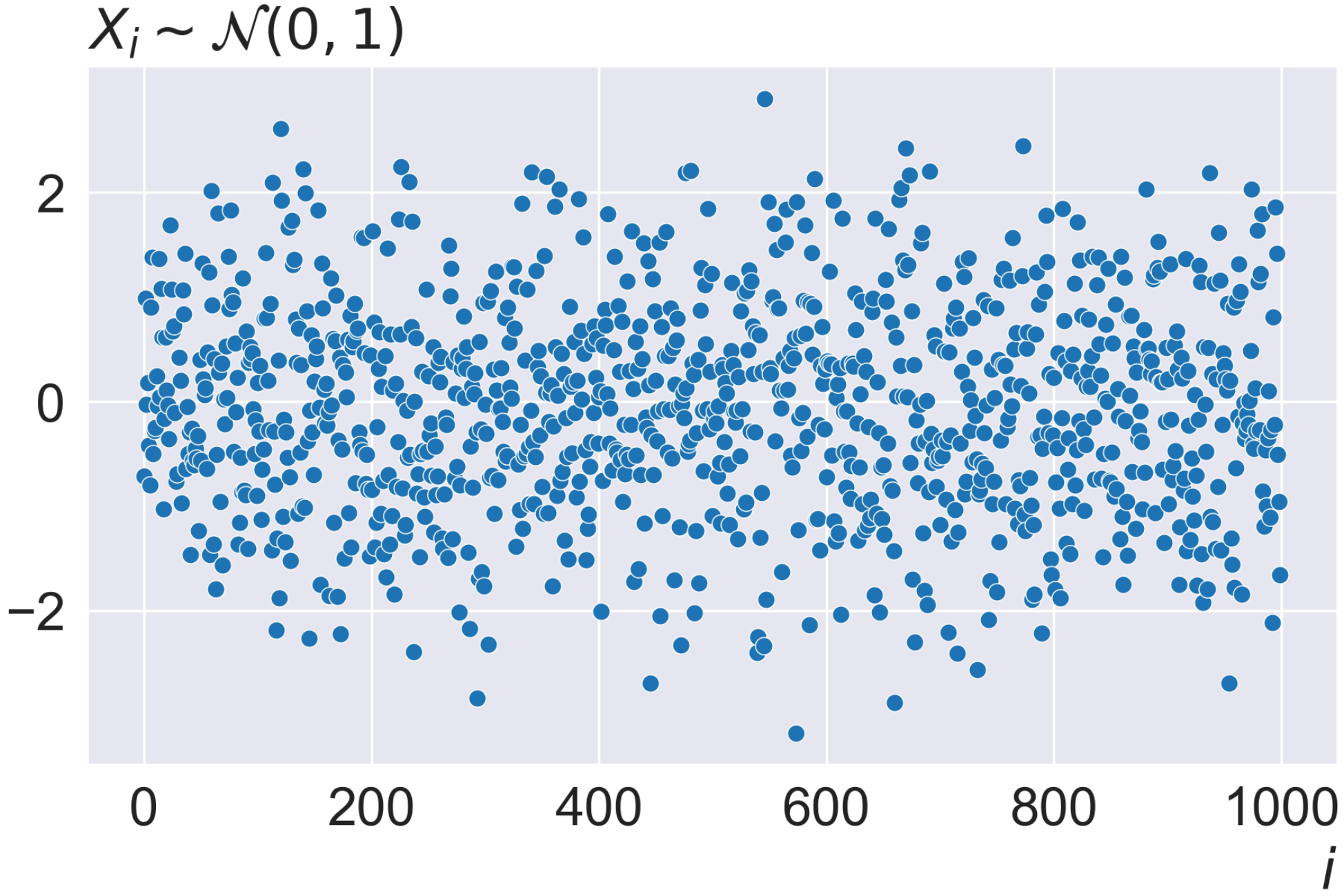
Let

$$U_1 \sim \mathcal{U}[0,1]$$
$$U_2 \sim \mathcal{U}[0,1]$$

be two independent simulations of standard uniform random variables. Then

$$N_1 = \sqrt{-2\ln(U_1)}\sin(2\pi U_2)$$
$$N_2 = \sqrt{-2\ln(U_1)}\cos(2\pi U_2)$$

are two independent simulations of standard Gaussian random variables.


$X_i \sim \mathcal{N}(0, 1)$
2
0
−2
0
200
400
600
800
1000
i

Lognormal distribution

If $X \sim \mathcal{N}(\mu, \sigma)$ is a normal random variable, then $Y = \exp(X) \sim \mathrm{LogNorm}(\mu, \sigma)$ follows a **lognormal distribution**, with probability distribution function given by

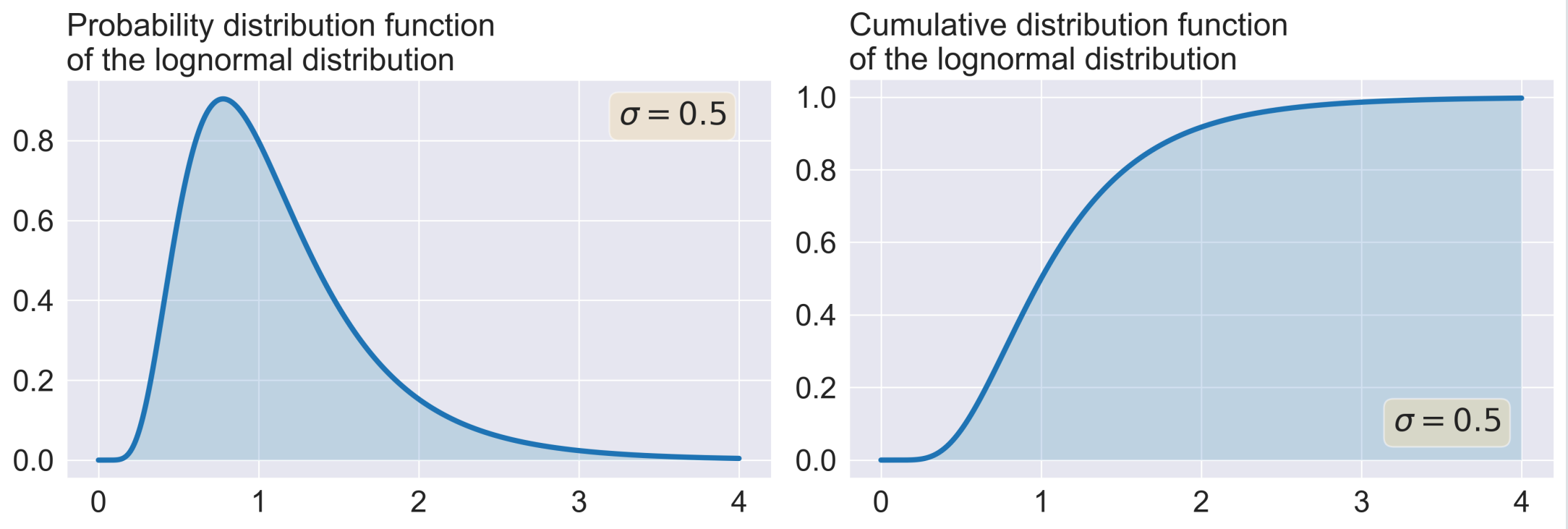
$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln(x) - \mu)^2}{2\sigma^2}\right)$$

and cumulative distribution function given by

$$F(x) = \Phi\left(\frac{\ln(x) - \mu}{\sigma}\right)$$

where Φ is the standard normal CDF.

Lognormal distribution

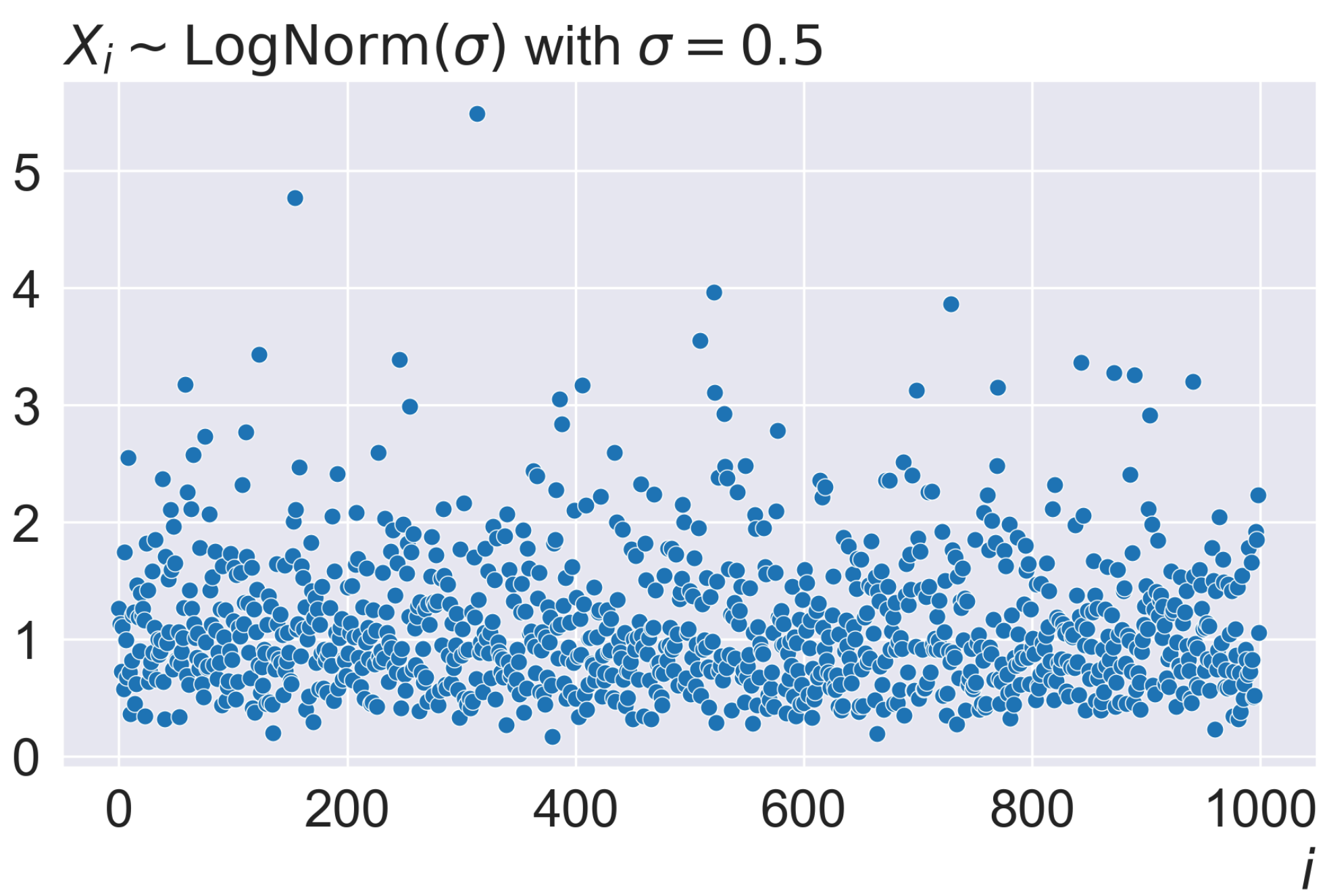


Simulate a lognormal random variable

Simulating a lognormal random variable is straightforward:

▷ Simulate a standard Gaussian variable $N \sim \mathcal{N}(0,1)$ using one of the algorithms described previously,
▷ Then $\exp(\mu + \sigma N)$ is a simulation of a lognormal variable with parameters μ and σ.

The lognormal distribution is used in finance to model asset prices at low frequency (months or higher). It provides a good fit for many other applications, such as the population size of cities (Gibrat's law), the length of chess games, or the length of comments posted in Internet discussion forums.


$X_i \sim \text{LogNorm}(\sigma)$ with $\sigma = 0.5$
5
4
3
2
1
0
0
200
400
600
800
1000
i

Exponential simulation with SciPy

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.expon.html

```
from scipy.stats import expon # exponential random variable
lmd = 2.0 # lambda parameter
expon.pdf([-0.25, 0.00, 1.00, 2.00], scale=1/lmd)
## array([0.        , 2.        , 0.27067057, 0.03663128])
expon.cdf([0.00, 0.50, 1.00, 2.00], scale=1/lmd)
## array([0.        , 0.63212056, 0.86466472, 0.98168436])
expon.ppf([0.00, 0.50, 0.90, 0.99], scale=1/lmd)
## array([0.        , 0.34657359, 1.15129255, 2.30258509])
expon.rvs(size=[2,4], scale=1/lmd)
## array([[0.42094583, 0.91815084, 0.00662086, 0.60169727],
##        [0.71008984, 0.25765254, 0.19886131, 1.28370504]])
```

Pareto simulation with SciPy

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pareto.html

```
from scipy.stats import pareto # pareto random variable
xm, alpha = 2.0, 2.0
pareto.pdf([1.0, 2.0, 3.00, 4.00], b=alpha, scale=xm)
## array([0.       , 1.       , 0.2962963, 0.125    ])
pareto.cdf([2.0, 3.0, 4.00, 5.00], b=alpha, scale=xm)
## array([0.        , 0.55555556, 0.75      , 0.84      ])
pareto.ppf([0.00, 0.50, 0.90, 0.99], b=alpha, scale=xm)
## array([ 2.        ,  2.82842712,  6.32455532, 20.        ])
pareto.rvs(size=[2,4], b=alpha, scale=xm)
## array([[2.73384241, 2.12184185, 2.10244472, 2.10286284],
##        [3.96741558, 6.24610087, 2.30588836, 3.22445486]])
```

Gaussian simulation with SciPy

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.norm.html

```
from scipy.stats import norm # normal random variable
mu, sigma = 2.0, 4.0
norm.pdf([-2.00, 2.00, 6.00, 10.00], loc=mu, scale=sigma)
## array([0.06049268, 0.09973557, 0.06049268, 0.01349774])
norm.cdf([-2.00, 2.00, 6.00, 10.00], loc=mu, scale=sigma)
## array([0.15865525, 0.5       , 0.84134475, 0.97724987])
norm.ppf([0.01, 0.50, 0.90, 0.99], loc=mu, scale=sigma)
## array([-7.3053915 ,  2.        ,  7.12620626, 11.3053915 ])
norm.rvs(size=[2,4], loc=mu, scale=sigma)
## array([[ 5.67561857,  4.38726532,  5.28242798, -0.63681343],
##        [-5.6528967 ,  3.72039664,  0.59963517, 11.18782141]])
```

Lognormal simulation with SciPy

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.lognorm.html

```
from math import exp
from scipy.stats import lognorm # lognormal random variable
mu, s = exp(2.0), 4.0
lognorm.pdf([0.0, mu, mu+s, mu+2*s], s=s, scale=mu)
## array([0.        , 0.01349774, 0.00870606, 0.00637284])
lognorm.cdf([0.0, mu, mu+10*s, mu+100*s], s=s, scale=mu)
## array([0.        , 0.5       , 0.67889026, 0.84193496])
lognorm.ppf([0.50, 0.90, 0.99], s=s, scale=mu)
## array([7.38905610e+00, 1.24414803e+03, 8.12585639e+04])
lognorm.rvs(size=[2,3], s=s, scale=mu)
## array([[1.55982314e+00, 1.56181214e+00, 3.16809476e+02],
##        [1.69722174e+03, 2.66809267e+01, 5.38547400e+00]])
```

4.3 Heavy-tailed distributions

Stylized facts

Many empirical studies have identified a set of common features among financial data that are known as **stylized facts**. These include:

▷ **Fat tails**: the unconditional distribution of returns has fatter tails than that expected from a normal distribution. This means that, if we use the normal distribution to model financial returns, we will underestimate the number and magnitude of crashes and booms.

▷ **Asymmetry**: the unconditional distribution is negatively skewed, suggesting that extreme negative returns are more frequent than extreme positive returns. The asymmetry and fat-tail phenomena persist even after adjusting for conditional *heteroskedasticity* (or changing volatility), meaning that the conditional distribution is also non-normal.

Stylized facts

▷ **Aggregated normality**: as the frequency of the returns lengthens, the return distribution gets closer to the normal distribution.

▷ **Absence of serial correlation**: returns generally do not display significant *serial correlation* (a.k.a. *autocorrelation*), except at high frequency. (These can be computed in Python with the Pandas function plotting.autocorrelation_plot).

▷ **Volatility clustering**: the volatility of returns is serially correlated, suggesting that a large (positive or negative) return tends to be followed by another large (positive or negative) return. Among the proxies for volatility, absolute returns appear to be the most strongly serially correlated.

Stylized facts

▷ **Time-varying cross-correlation**: correlation between asset returns tends to increase during high volatility periods, in particular during crashes.

[1] R. Cont. "Empirical properties of asset returns: stylized facts and statistical issues". In: *Quantitative Finance* 1.2 (2001), pp. 223-236.

[2] A. Chakraborti, I. Muni Toke, M. Patriarca, et al. "Econophysics review: I. Empirical facts". In: *Quantitative Finance* 11.7 (2011), pp. 991-1012.

[3] E. Ratliff-Crain, C. Van Oort, J. Bagrow, et al. "Revisiting stylized facts for modern stock markets". In: *IEEE International Conference on Big Data*. 2023, pp. 1814-1823.

Student's t-distribution

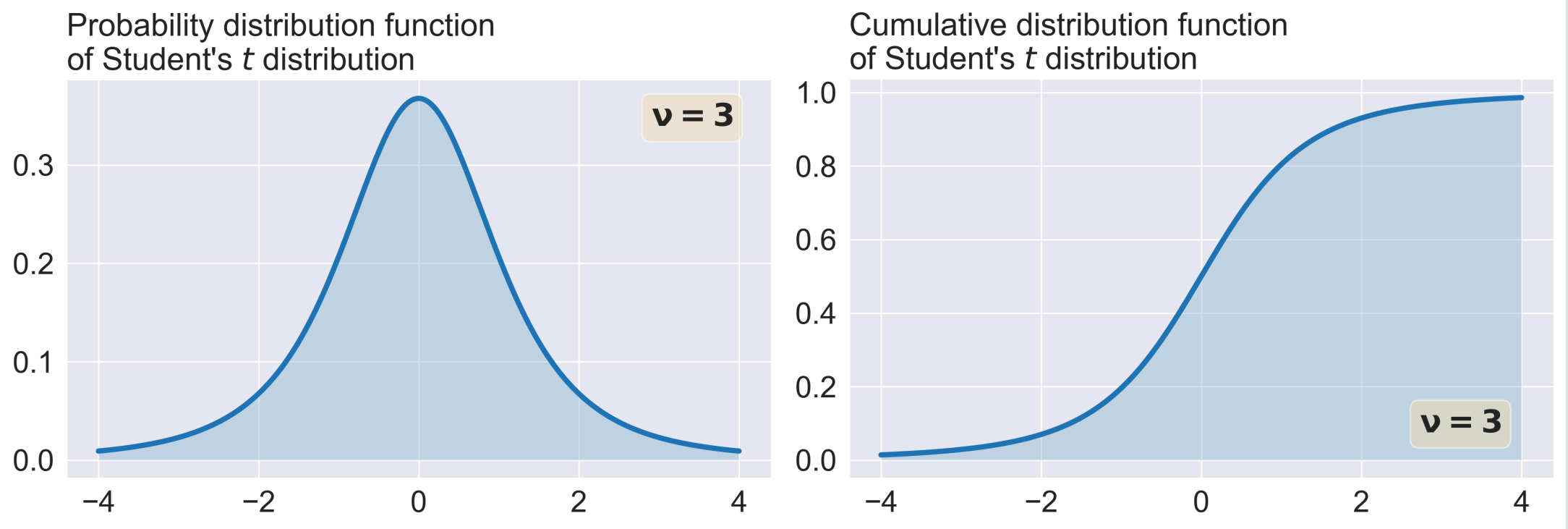
The **Student t-distribution** t_ν with $\nu > 0$ degrees of freedom is defined by the PDF

$$f(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)}\left(1+\frac{x^2}{\nu}\right)^{-(\nu+1)/2},$$

where Γ is the Gamma function $\Gamma(z) = \int_0^\infty t^{z-1}e^{-t}dt$. When n is a positive integer, $\Gamma(n) = (n-1)!$

The CDF of the Student t-distribution is semi-analytical in terms of the ${}_2F_1$ hypergeometric function.

Student's t-distribution



Student's t-distribution

▷ When $\nu \to \infty$, then the Student t distribution converges to the Gaussian distribution.

▷ When $\nu < \infty$, the Student t distribution has *heavier tails* than the Gaussian distribution. Indeed, the **moments** $\mathbb{E}[X^p]$ of a t-distributed random variable X only exist for $p < \nu$.

▷ In finance, the generalized Student t-distribution $X = \mu + \sigma T$ where μ is a location parameter, σ is a scale parameter, and T is a standard t-distributed random variable with $\nu \approx 4$ has been shown to provide a very good fit for the log-returns of many worldwide stock indices.

Simulate a t-distributed random variable

Let

$$U_1 \sim \mathcal{U}[0,1]$$
$$U_2 \sim \mathcal{U}[0,1]$$

be two independent simulations of standard uniform random variables. Then

$$T_1 = \sqrt{\nu(U_1^{-2/\nu} - 1)}\sin(2\pi U_2)$$
$$T_2 = \sqrt{\nu(U_1^{-2/\nu} - 1)}\cos(2\pi U_2)$$

are two simulations of t-distributed random variables with ν degrees of freedom. Unfortunately T_1 and T_2 are **not** independent.

Simulate a t-distributed random variable

When $\nu \to \infty$, we recover the **Box-Muller algorithm** and the Student t-distribution becomes a Gaussian distribution.

This extension of the Box-Muller algorithm to Student's t-distribution was discovered in:

[1] R. Bailey. "Polar generation of random variates with the t-distribution". In: *Mathematics of Computation* 62.206 (1994), pp. 779-781.

This article also describes a slightly faster version of this simulation algorithm, which uses the **acceptance-rejection method**. It is described in the next slide.

Simulate a t-distributed random variable

Let $U_1 \sim \mathcal{U}[0,1]$ and $U_2 \sim \mathcal{U}[0,1]$ be two independent simulations of standard uniform random variables. Compute

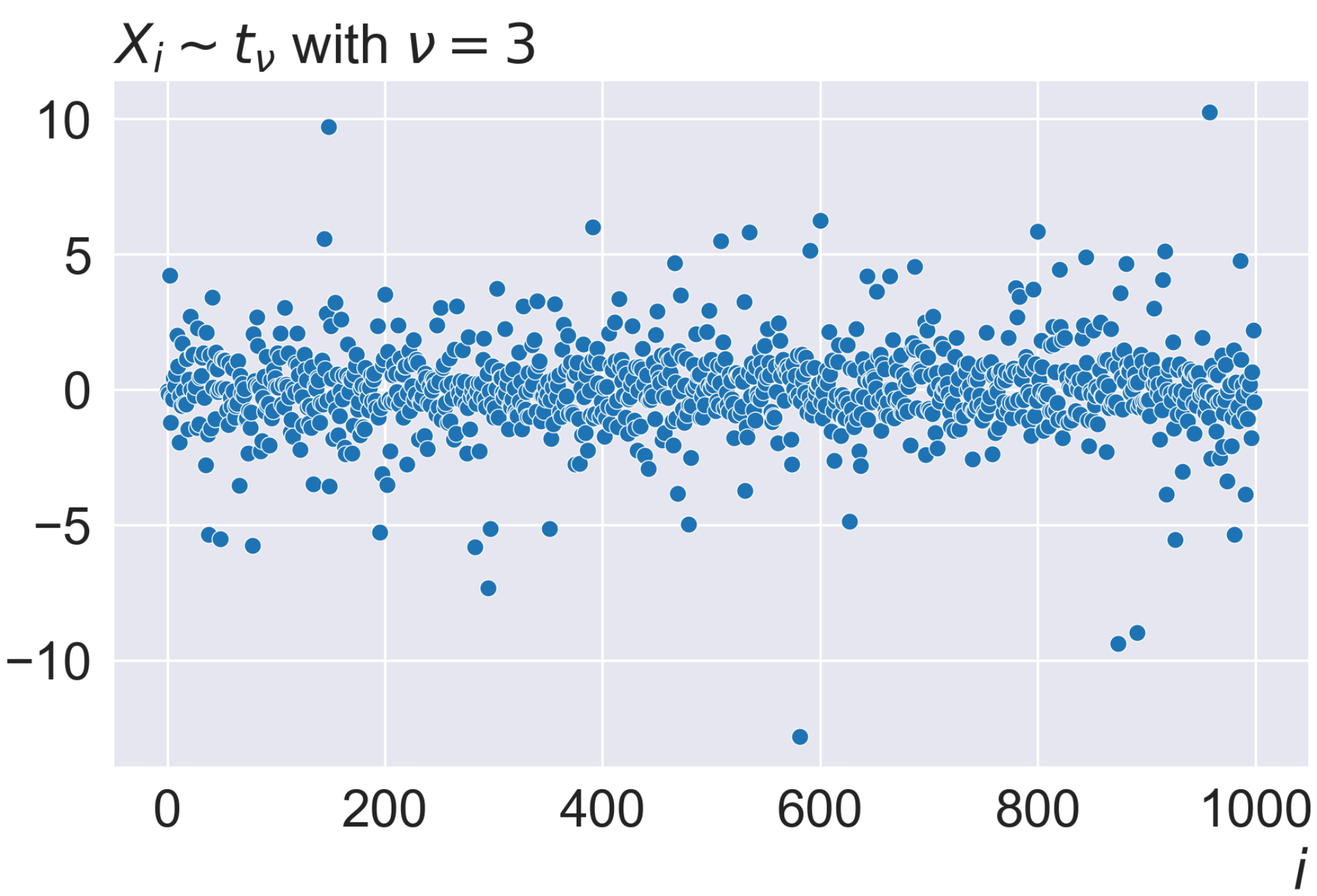
$$W = (2U_1 - 1)^2 + (2U_2 - 1)^2$$

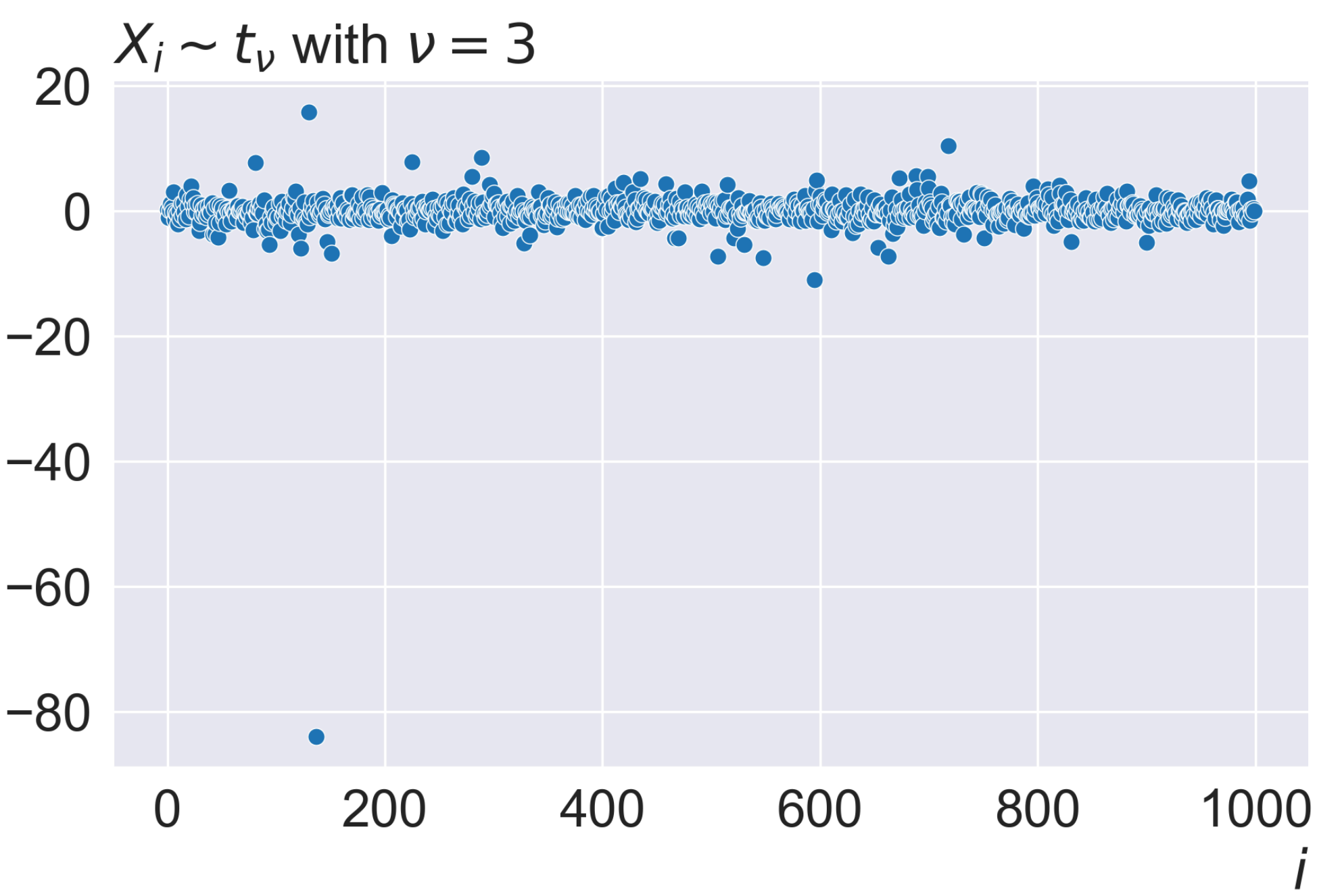
As long as $W > 1$, **resimulate** U_1 and U_2 until $W \leq 1$. Then,

$$T_1 = \sqrt{\nu(W^{-2/\nu} - 1)U_1^2/W}$$

$$T_2 = \sqrt{\nu(W^{-2/\nu} - 1)U_2^2/W}$$

are t-distributed with ν degrees of freedom. Like before, T_1 and T_2 are not independent.


$X_i \sim t_\nu$ with $\nu = 3$
10
5
0
−5
−10
0
200
400
600
800
1000
i


$X_i \sim t_\nu$ with $\nu = 3$
20
0
−20
−40
−60
−80
0
200
400
600
800
1000
i

Cauchy distribution

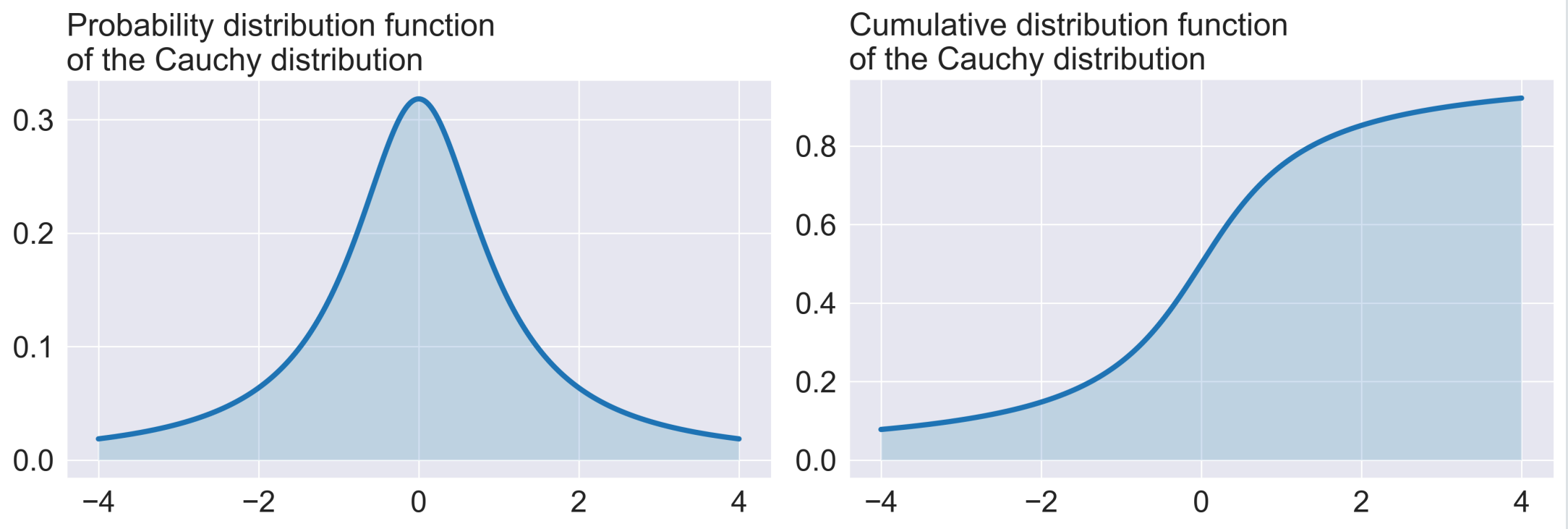
The **Cauchy distribution** is defined by the PDF

$$f(x) = \frac{1}{\pi\gamma\left[1 + \left(\frac{x-x_0}{\gamma}\right)^2\right]},$$

where x_0 is a location parameter and γ is a scale parameter. It is equivalently defined by the CDF

$$F(x) = \frac{1}{2} + \frac{1}{\pi}\arctan\left(\frac{x - x_0}{\gamma}\right)$$

Cauchy distribution



Cauchy distribution

▷ The standard Cauchy distribution with $x_0 = 0$ and $\gamma = 1$ corresponds to the Student t-distribution with $\nu = 1$. In particular, the moments $\mathbb{E}[X^p]$ of a Cauchy random variable X only exist for $p < 1$.

▷ This means that **none of the positive integer moments of the Cauchy distribution exist**, not even the variance, not even the mean! Because of that, various results in probability theory about expected values, such as the strong law of large numbers, fail to hold for the Cauchy distribution.

▷ The family of Student's t-distributions interpolates between the well-behaved Gaussian distribution (for $\nu = \infty$) and the extremely heavy-tailed Cauchy distribution (for $\nu = 1$).

Simulate a Cauchy random variable

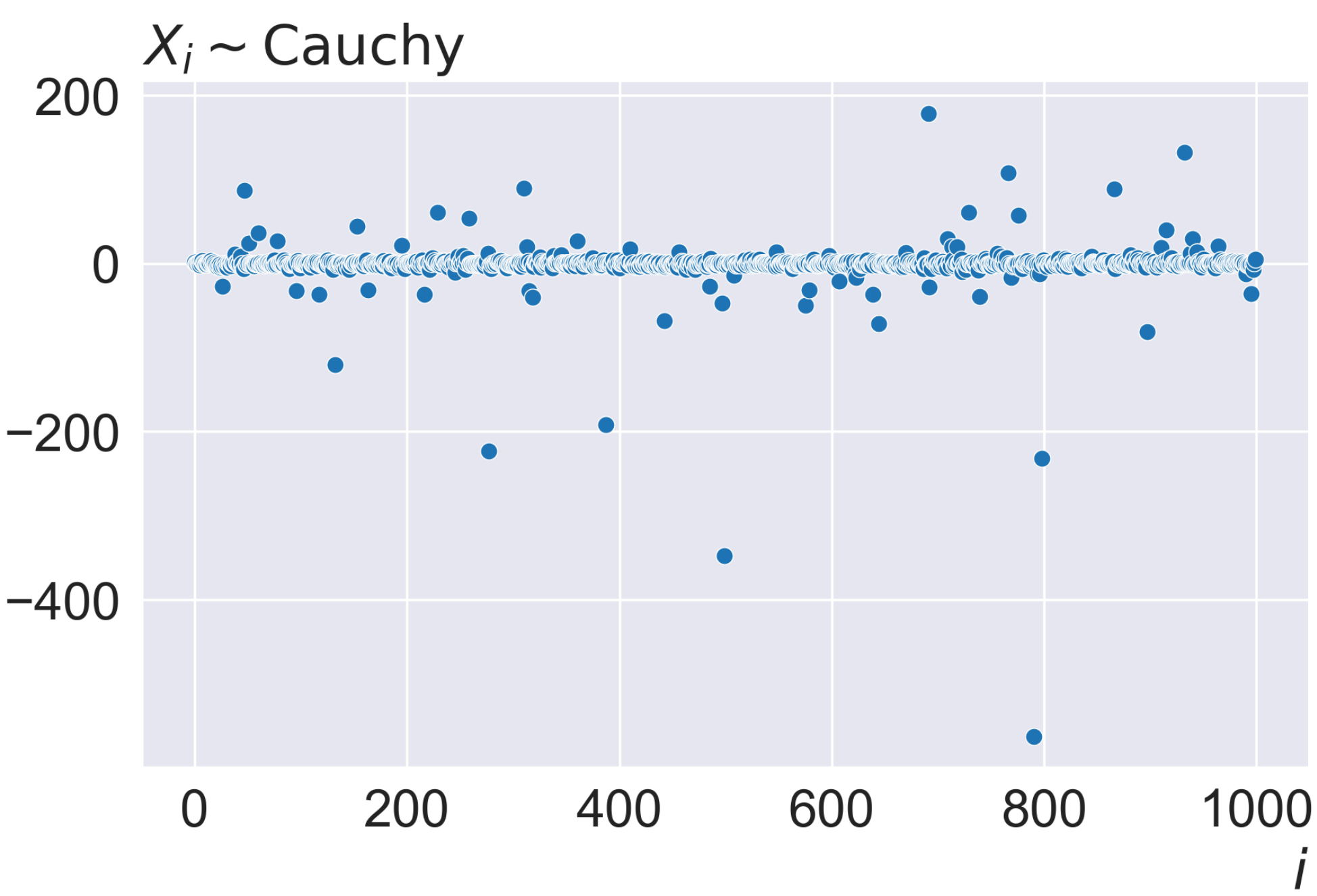
A Cauchy distributed random variable can be simulated by the **inversion method**. If U is a standard uniform random variable, then

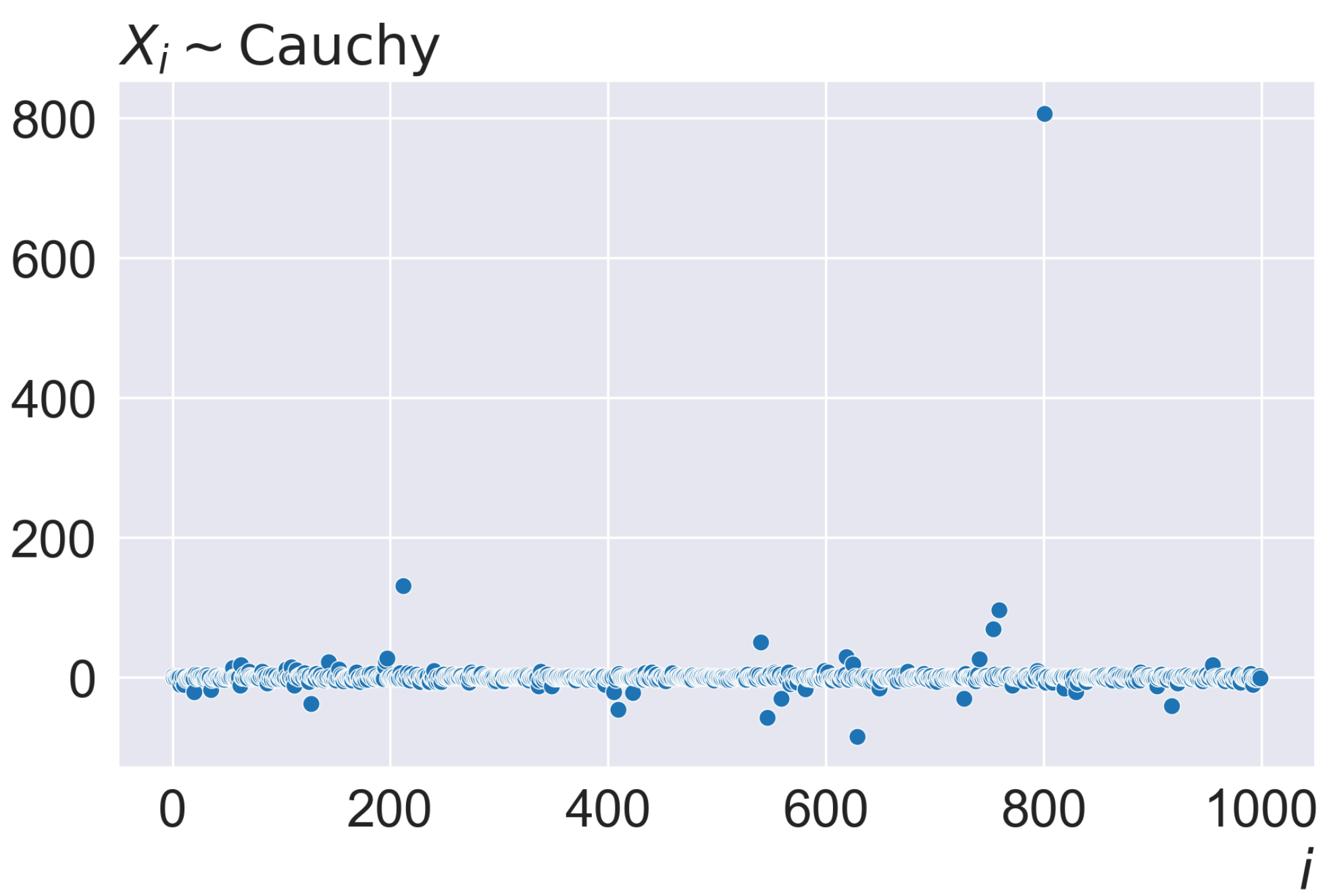
$$F^{-1}(U) = x_0 + \gamma \tan\left[\pi\left(U - \frac{1}{2}\right)\right]$$

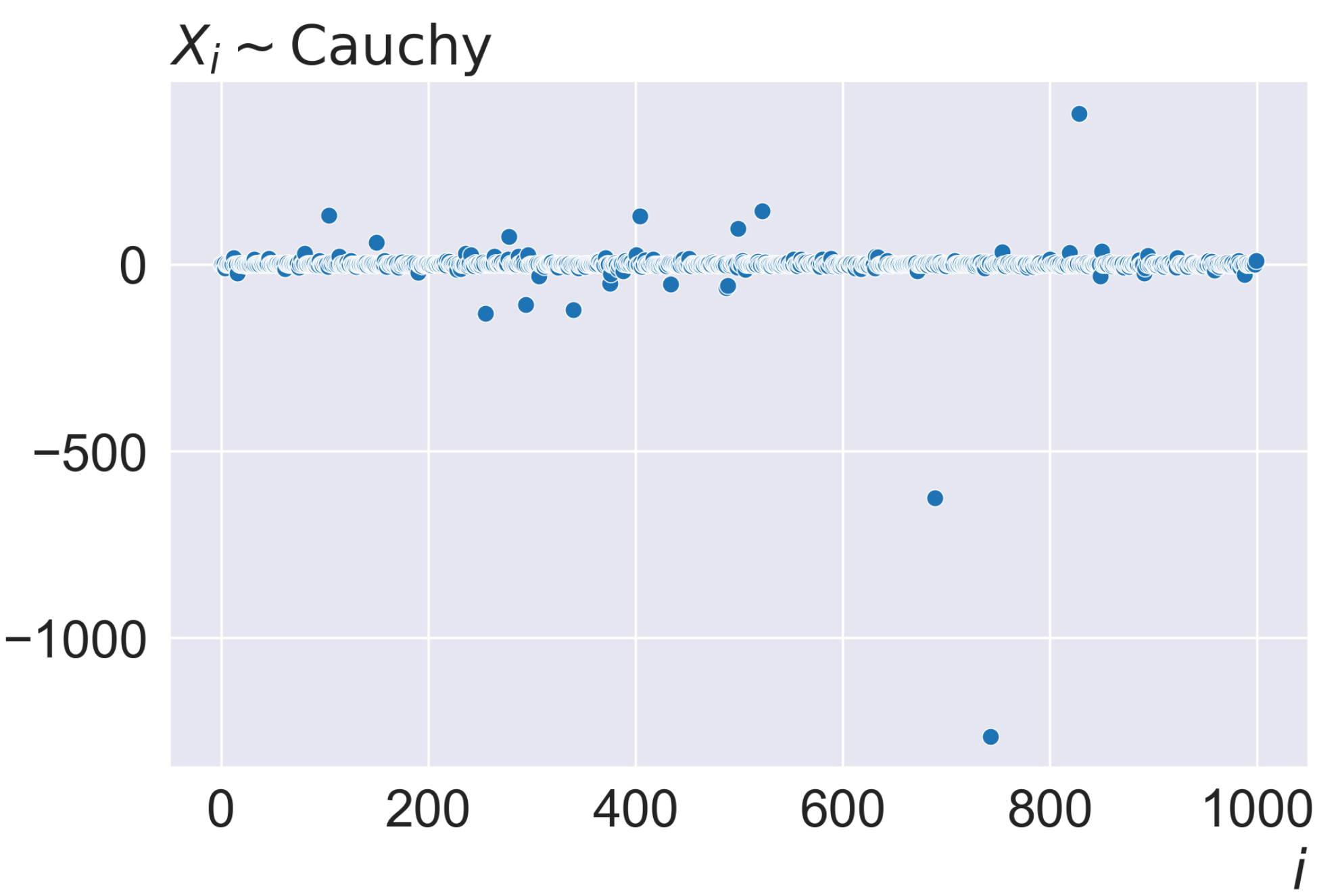
is Cauchy-distributed with parameters x_0 and γ. It can also be simulated using Bailey's method for the Student t-distribution with $\nu = 1$

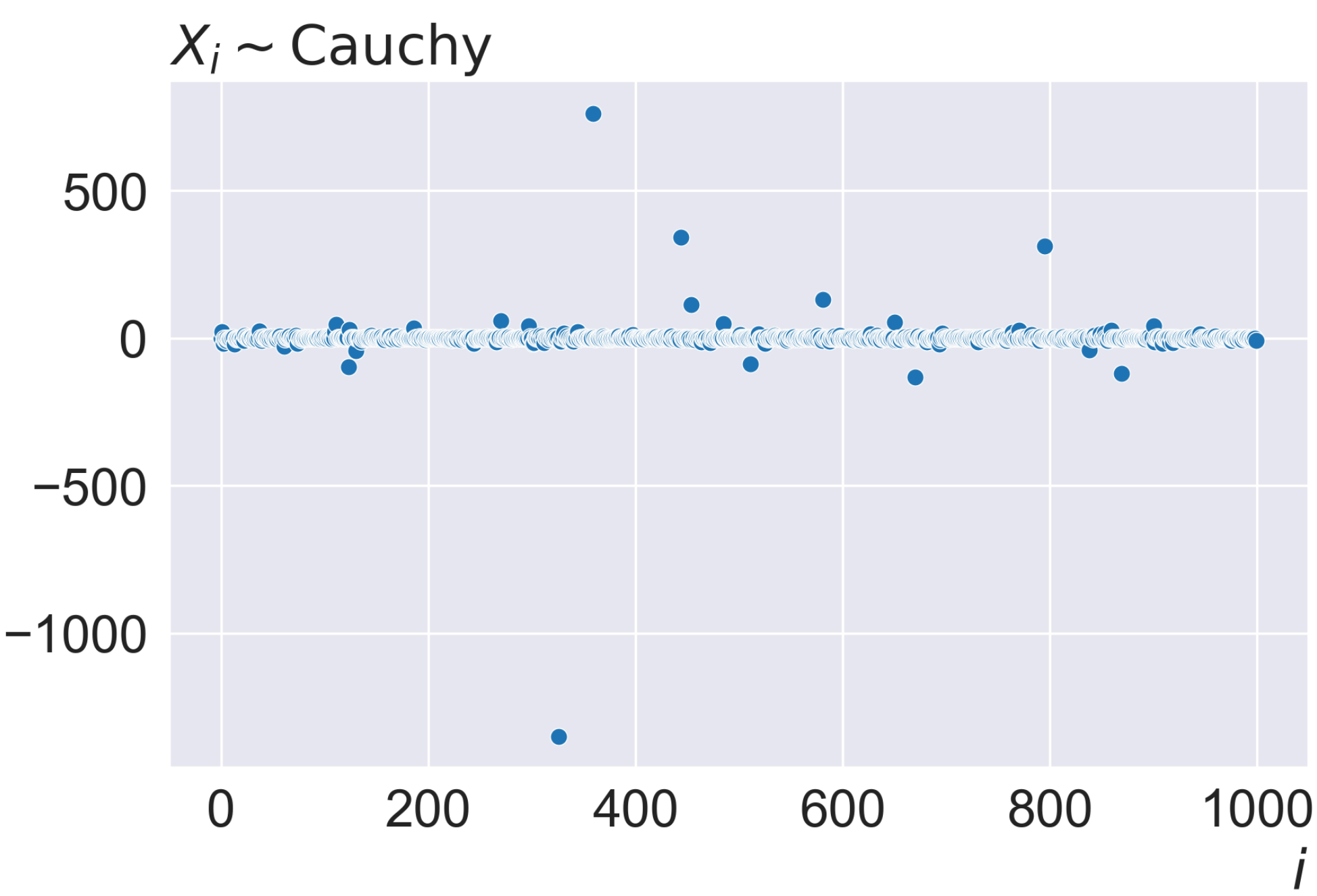
$$x_0 + \gamma\sqrt{\frac{1}{U_1^2} - 1}\sin(2\pi U_2)$$

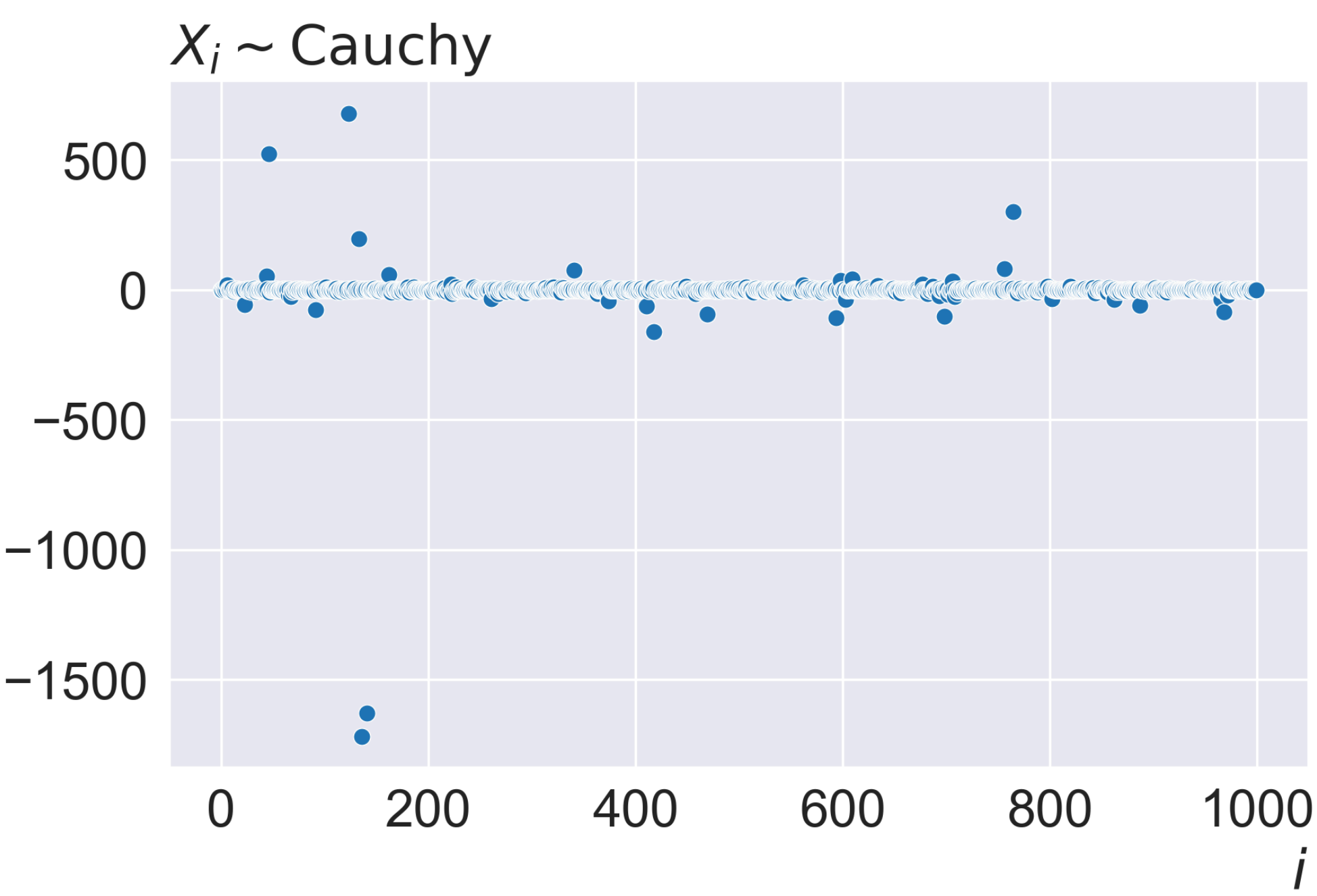
where U_1 and U_2 are two i.i.d. standard uniform random variables.

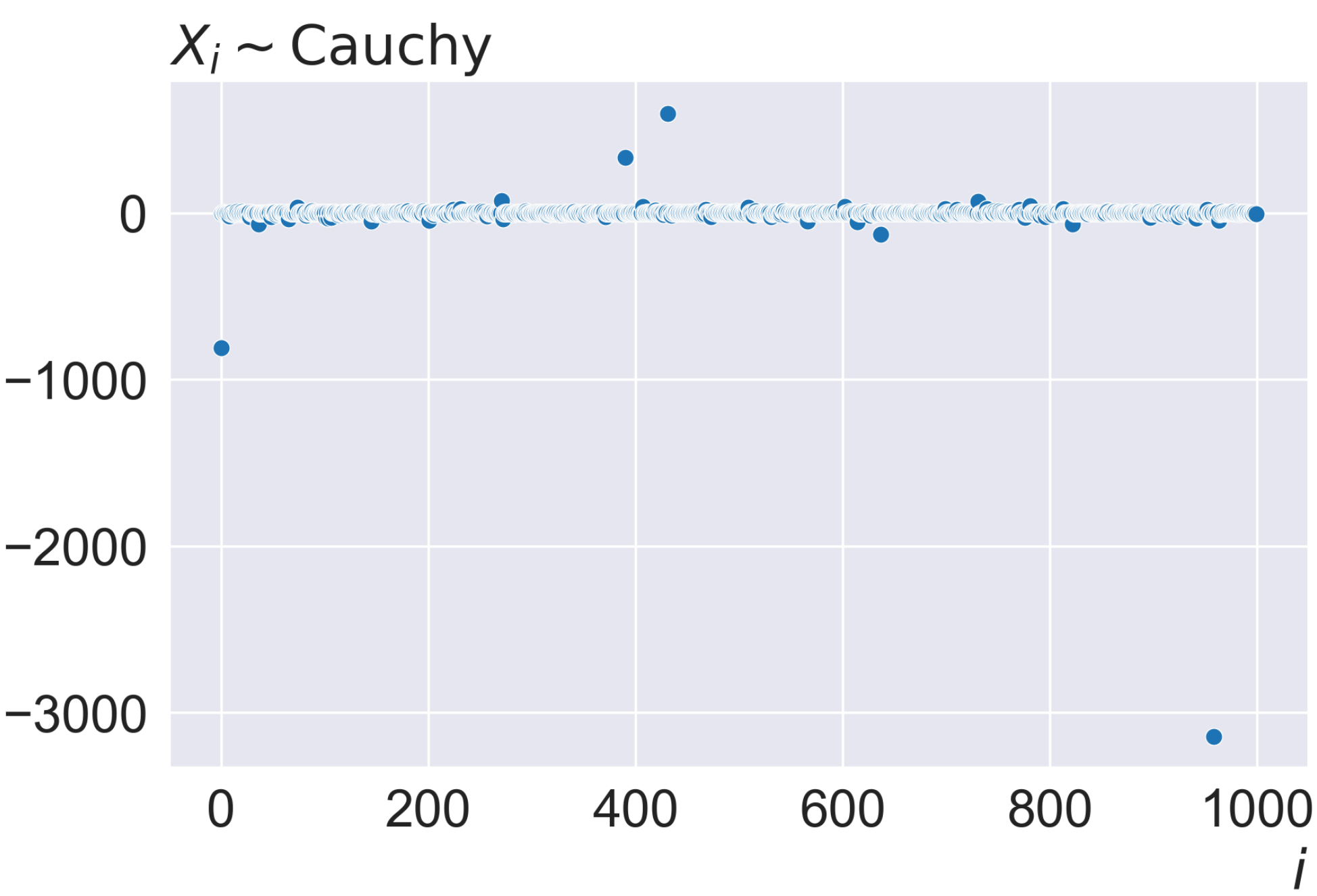

$X_i \sim$ Cauchy
200
0
−200
−400
0
200
400
600
800
1000
i

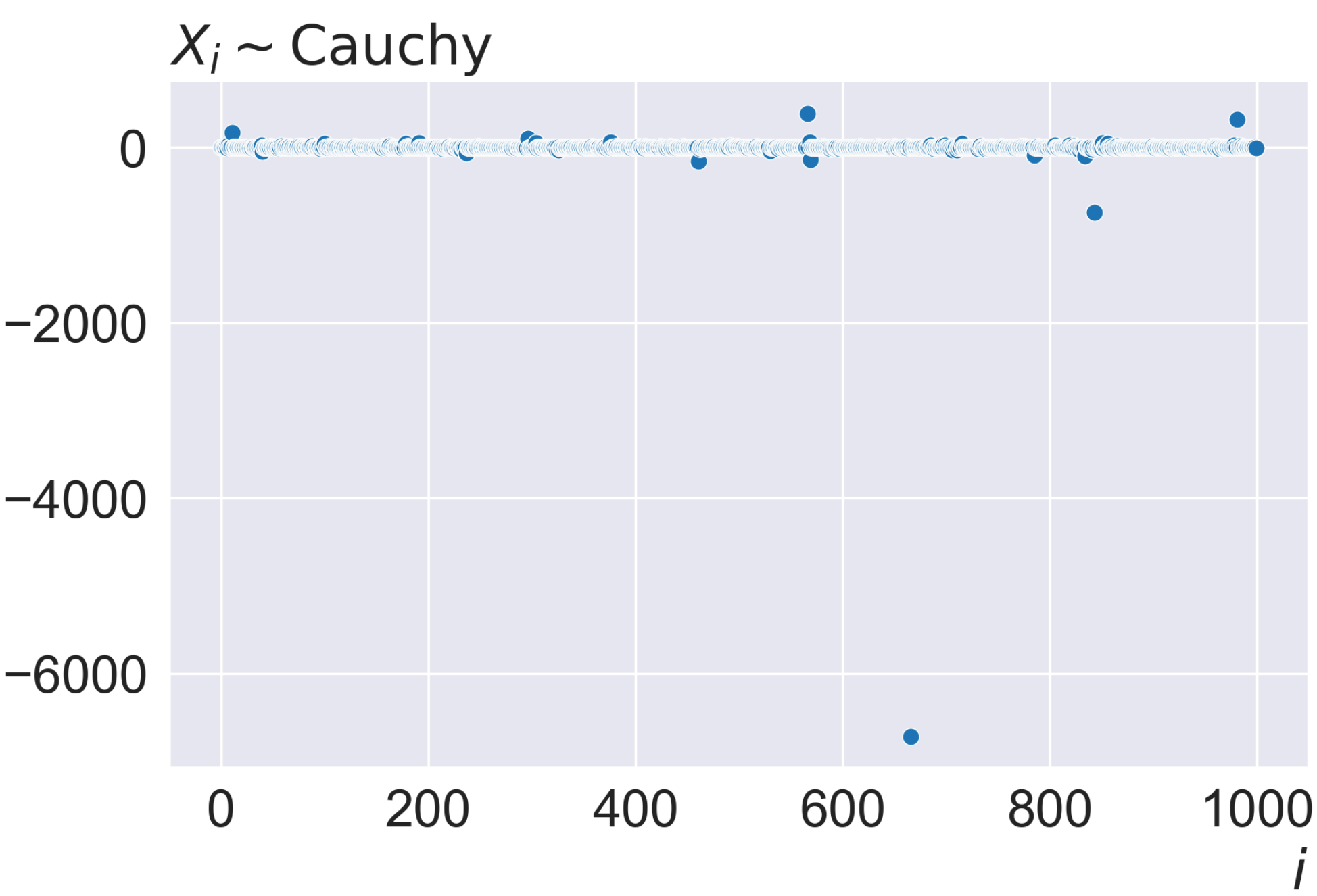

$X_i \sim$ Cauchy
800
600
400
200
0
0
200
400
600
800
1000
i

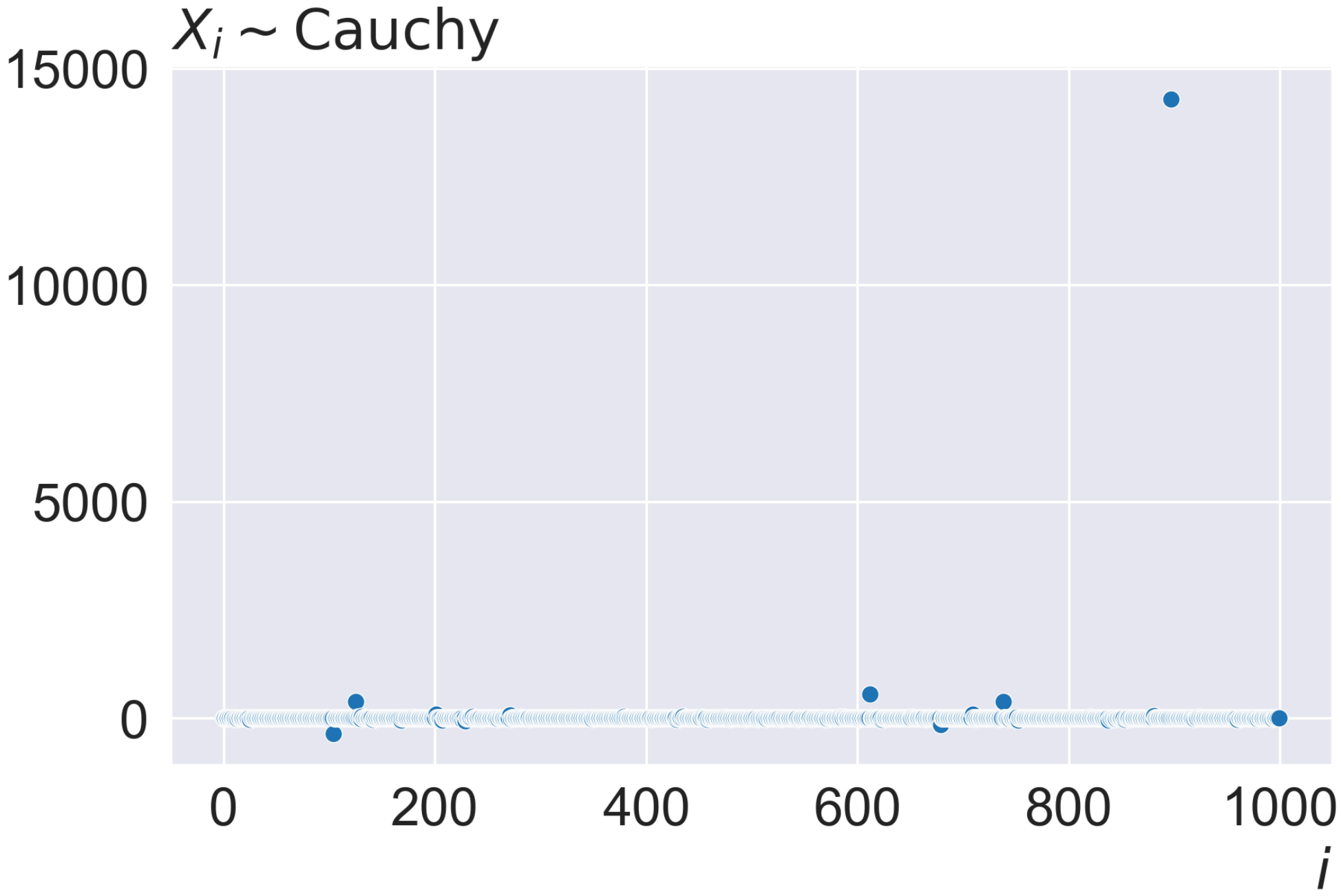

$X_i \sim$ Cauchy
0
−500
−1000
0
200
400
600
800
1000
i

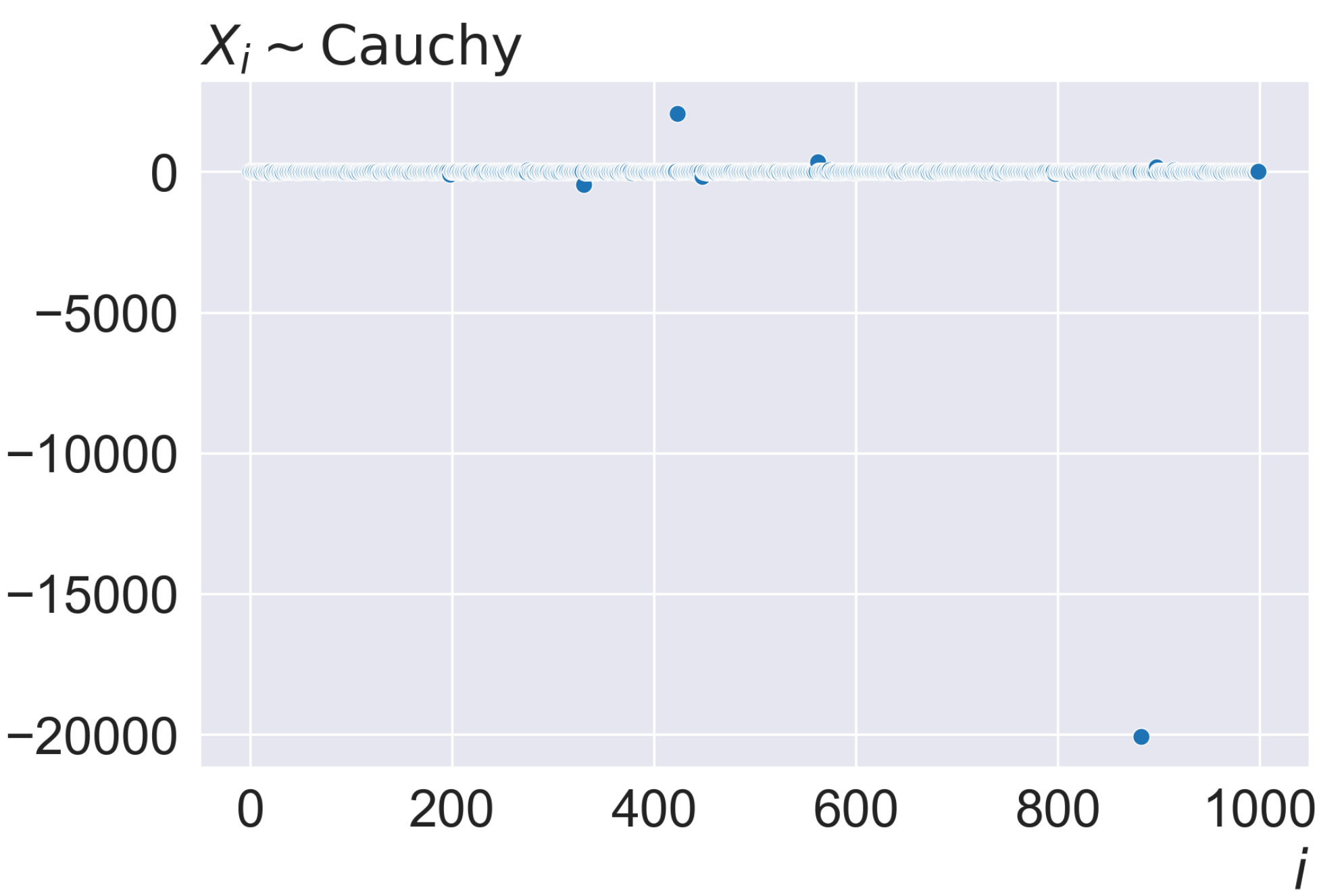

$X_i \sim$ Cauchy
500
0
−500
−1000
0
200
400
600
800
1000
i

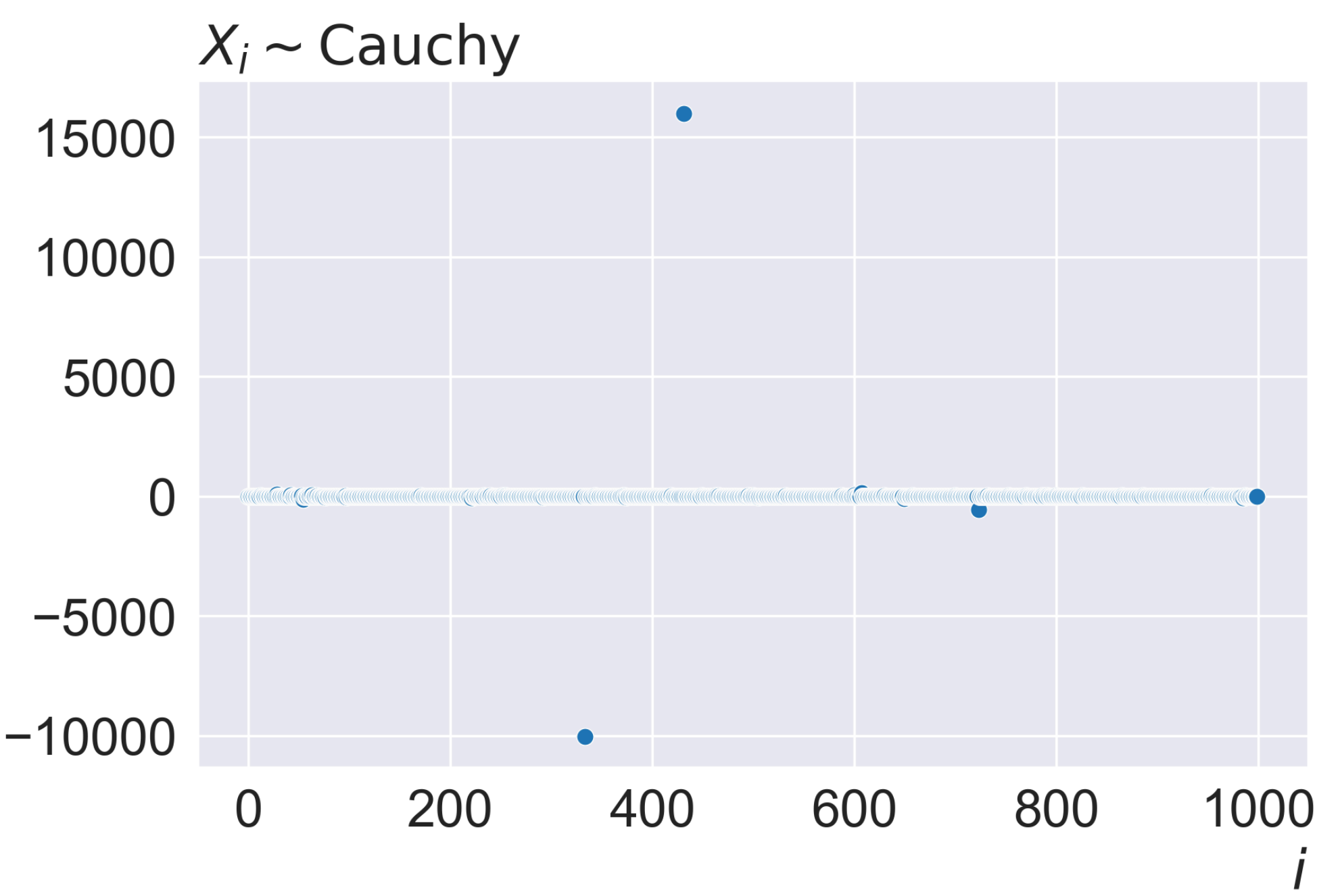

$X_i \sim$ Cauchy
500
0
−500
−1000
−1500
0
200
400
600
800
1000
i


$X_i \sim$ Cauchy
0
−1000
−2000
−3000
0
200
400
600
800
1000
i


$X_i \sim$ Cauchy
0
−2000
−4000
−6000
0
200
400
600
800
1000
i


$X_i \sim$ Cauchy
15000
10000
5000
0
0
200
400
600
800
1000
i


$X_i \sim$ Cauchy
0
−5000
−10000
−15000
−20000
0
200
400
600
800
1000
i


$X_i \sim$ Cauchy
15000
10000
5000
0
−5000
−10000
0
200
400
600
800
1000
i

Other distributions of interest in finance

Chi-squared distribution $\chi^2(k)$

$$f(x) = \frac{1}{2^{k/2}\Gamma(k/2)} x^{k/2-1} e^{-x/2}$$

Generalized extreme value distribution $\mathrm{GEV}(\mu, \sigma, \xi)$, which combines the **Gumbel**, **Fréchet** and **Weibull** families of distributions.

$$f(x) = \tfrac{1}{\sigma} t(x)^{\xi+1} e^{-t(x)} \quad \text{where} \quad t(x) = \begin{cases} \left(1 + \xi\left(\frac{x-\mu}{\sigma}\right)\right)^{-1/\xi} & \text{if } \xi \neq 0 \\ \exp\left(-\frac{x-\mu}{\sigma}\right) & \text{if } \xi = 0 \end{cases}$$

Student's t simulation with SciPy

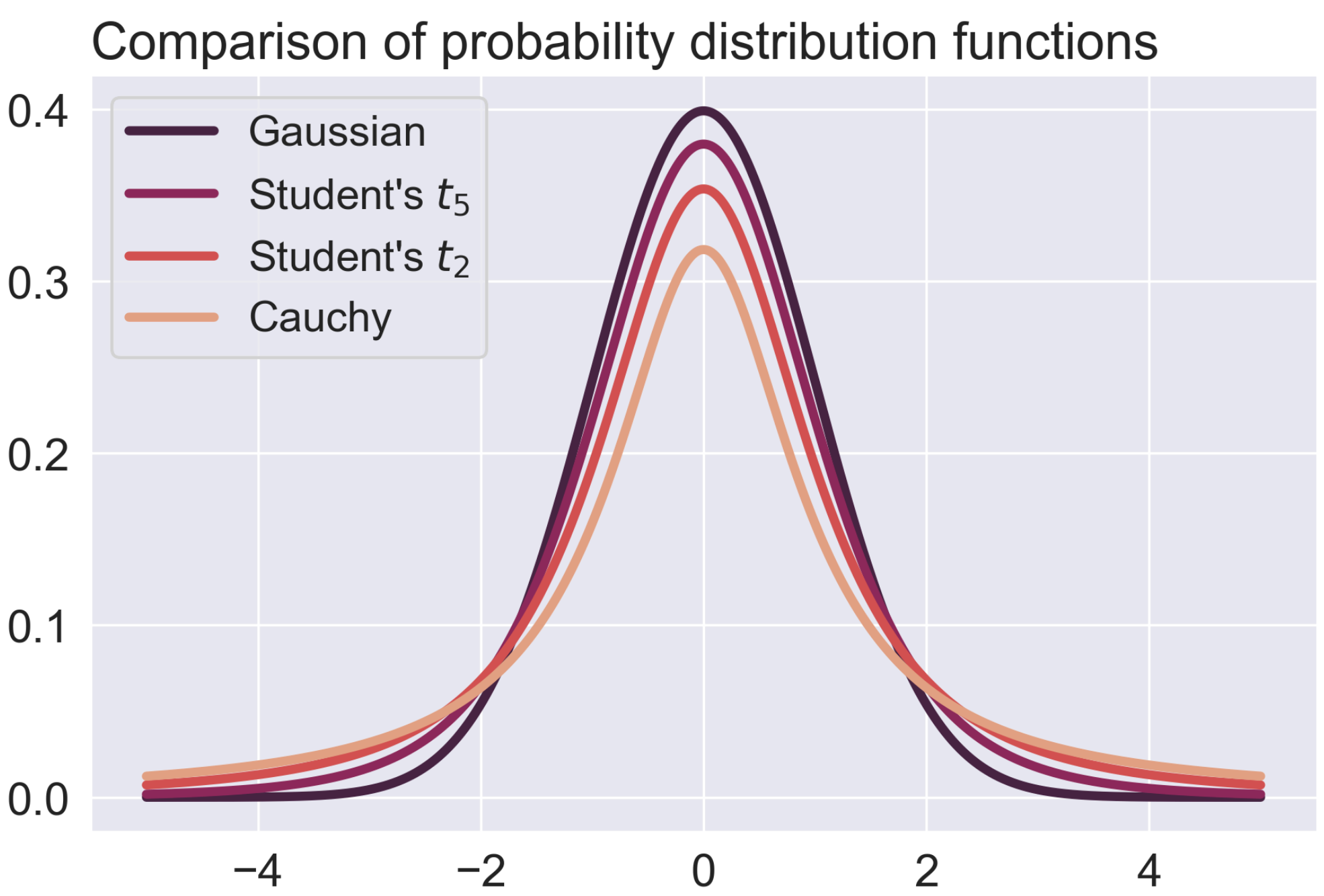
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.t.html

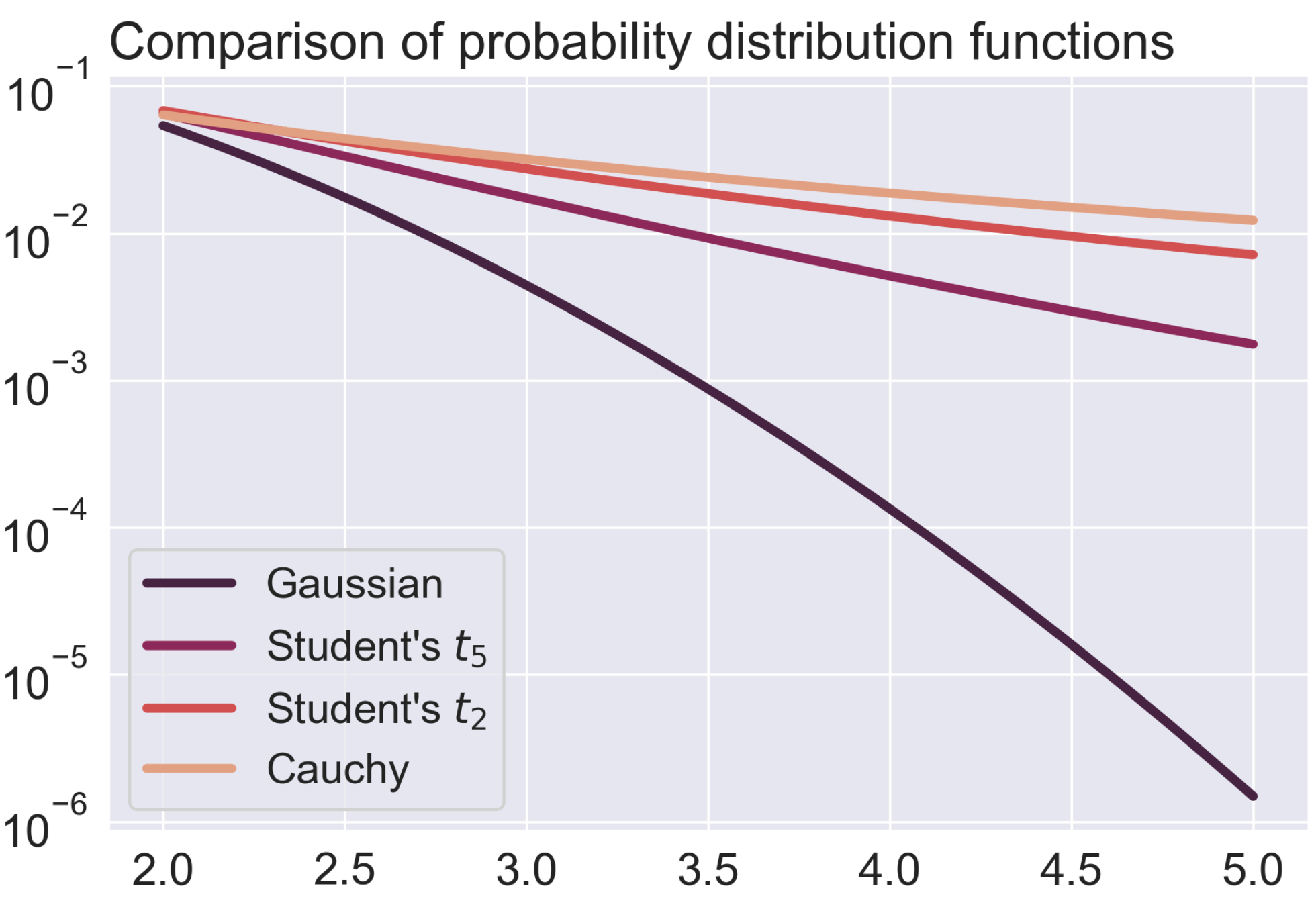
```
from scipy.stats import t # t random variable
nu = 3
t.pdf([-4, 0 ,2, 4], df=nu)
## array([0.00916336, 0.3675526 , 0.06750966, 0.00916336])
t.cdf([-4, 0 ,2, 4], df=nu)
## array([0.01400423, 0.5       , 0.93033702, 0.98599577])
t.ppf([0.10, 0.50, 0.99], df=nu)
## array([-1.63774436e+00,  7.20376043e-17,  4.54070286e+00])
t.rvs(size=[2,4], df=nu)
## array([[-0.36100706, -5.12578936, -0.32755638, -0.88675497],
##        [ 0.67829222, -0.3251135 , -2.82284145, -0.02353942]])
```

Cauchy simulation with SciPy

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.cauchy.html

```
from scipy.stats import cauchy # cauchy random variable
cauchy.pdf([-4, 0 ,2, 4])
## array([0.01872411, 0.31830989, 0.06366198, 0.01872411])
cauchy.cdf([-4, 0 ,2, 4])
## array([0.07797913, 0.5       , 0.85241638, 0.92202087])
cauchy.ppf([0.10, 0.50, 0.99])
## array([-3.07768354,  0.        , 31.82051595])
cauchy.rvs(size=[2,4])
## array([[ 3.70319255, -0.29676117,  1.37436795,  0.28676777],
##        [-3.94540283,  3.61338681,  0.62736869,  0.77810087]])
```


Comparison of probability distribution functions
Gaussian
Student's t_5
Student's t_2
Cauchy
0.4
0.3
0.2
0.1
0.0
−4
−2
0
2
4


Comparison of probability distribution functions
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^{-6}
Gaussian
Student's t_5
Student's t_2
Cauchy
2.0
2.5
3.0
3.5
4.0
4.5
5.0